



# Estilos y Patrones III

IIC2173 - Arquitectura de sistemas de software

Departamento de Ciencia de la Computación

Escuela de Ingeniería – PUC

Nicolás Acosta – Gerardo Crot

# Repaso

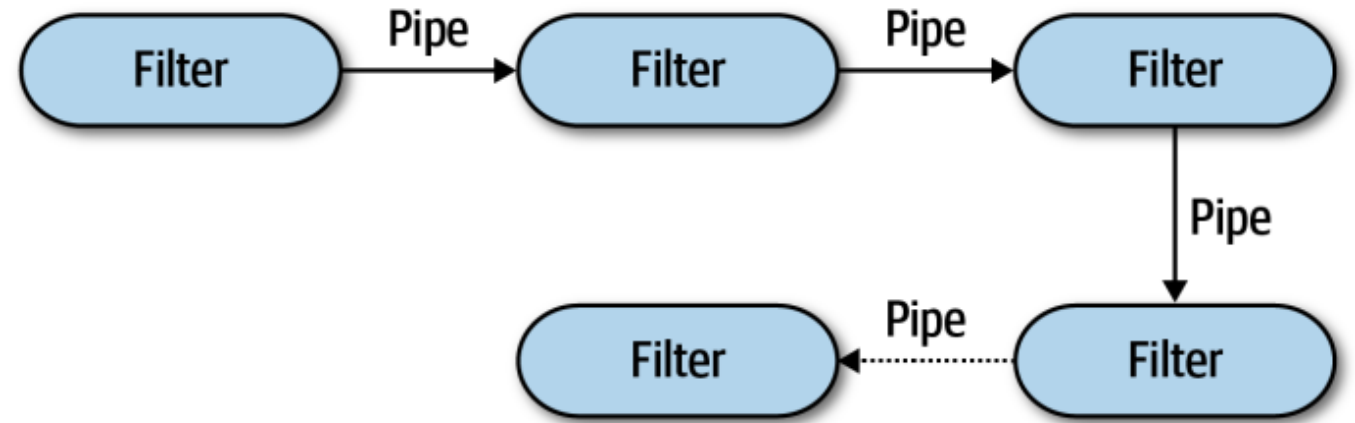
---

- Main y subrutinas
- En capas
  - Cliente Servidor
  - Máquina Virtual
- MicroKernel
- Flujo de datos:
  - En lotes secuenciales
  - Pipe & Filter



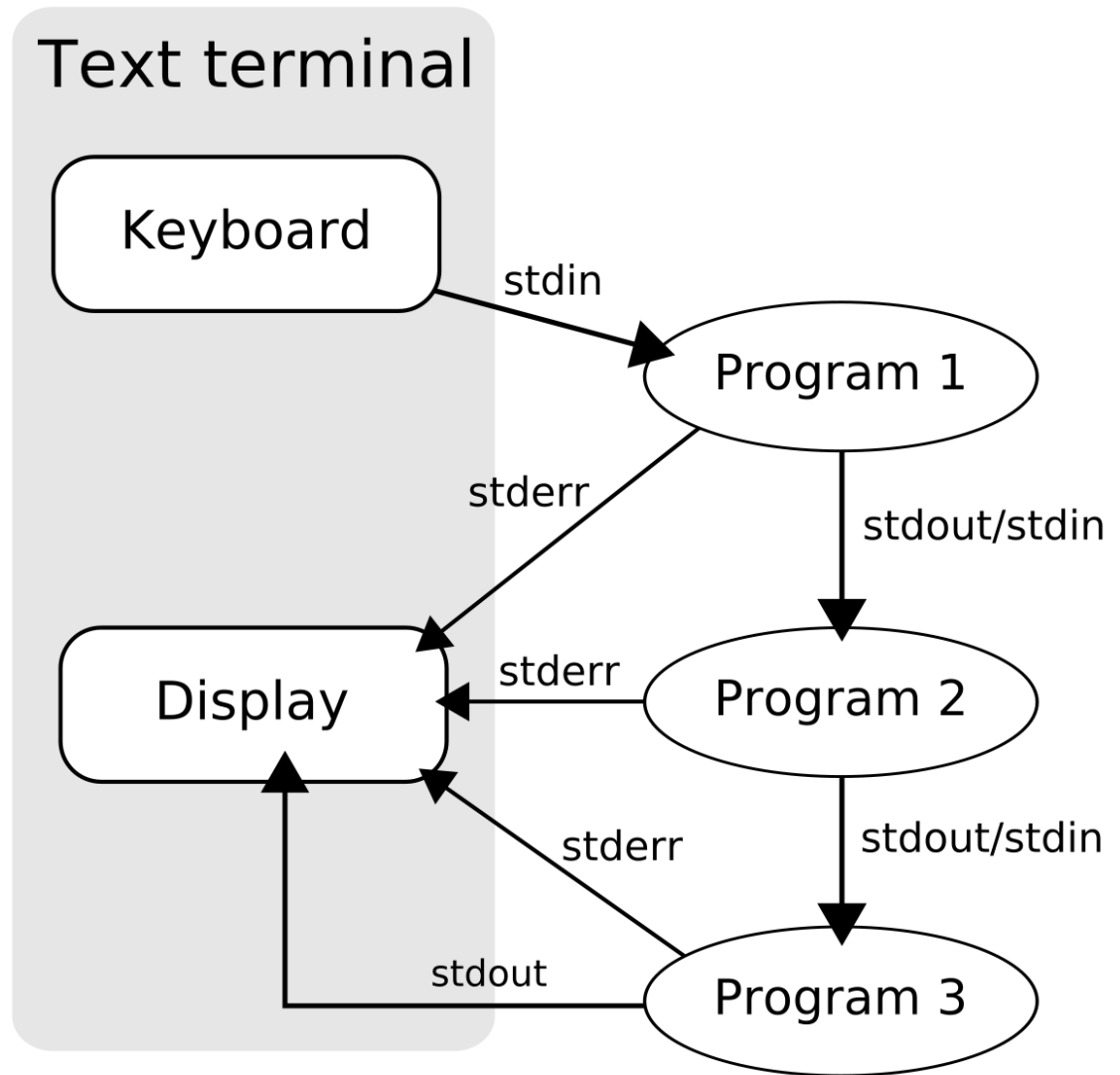
# Flujo de datos: Pipe and Filter

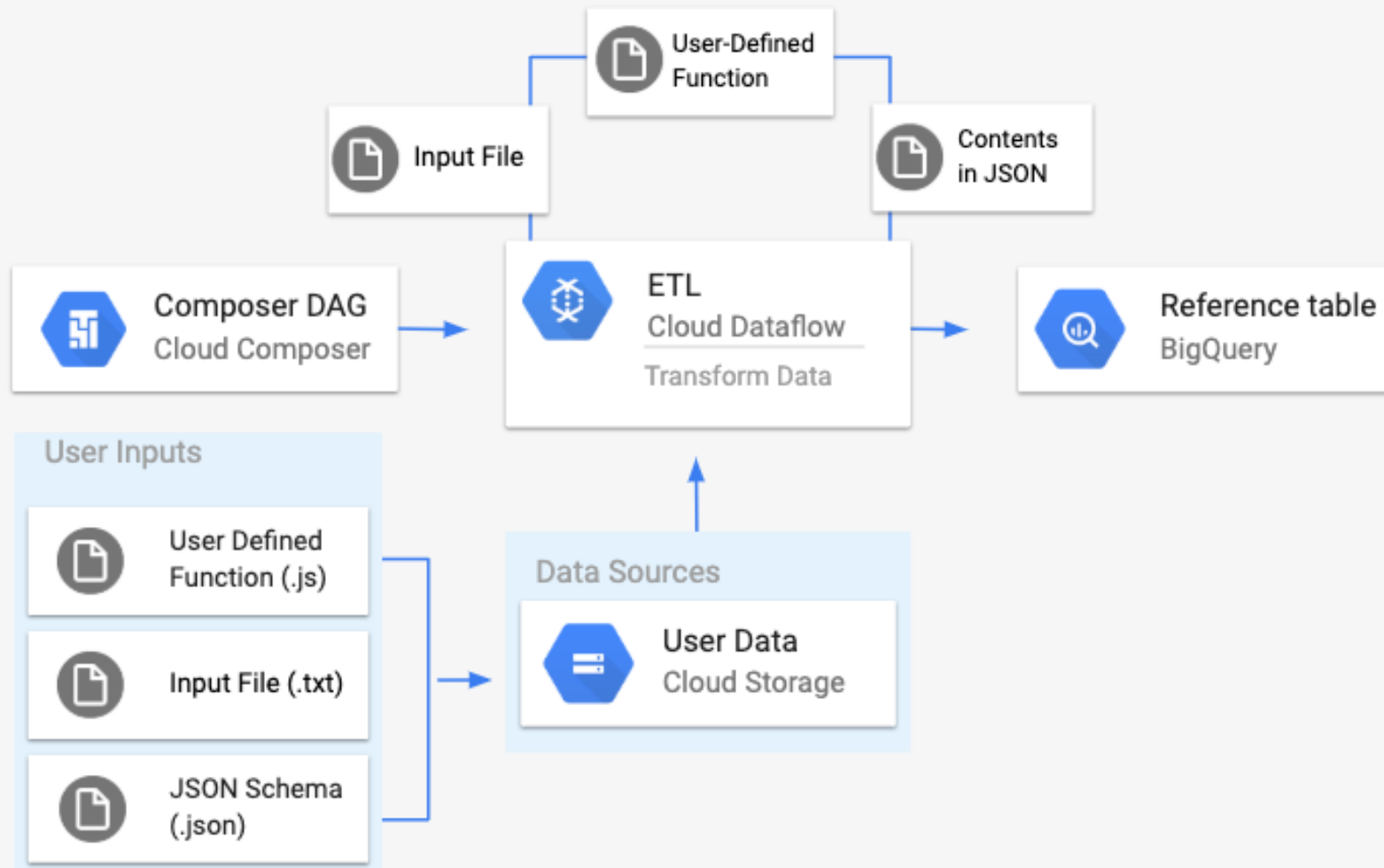
- Estilo sumamente poderoso
- Componentes con **interfaces claras**
- Típicamente los pipes son unidireccionales y de punto a punto
- Cada componente Filtro **cumple una función encapsulada**

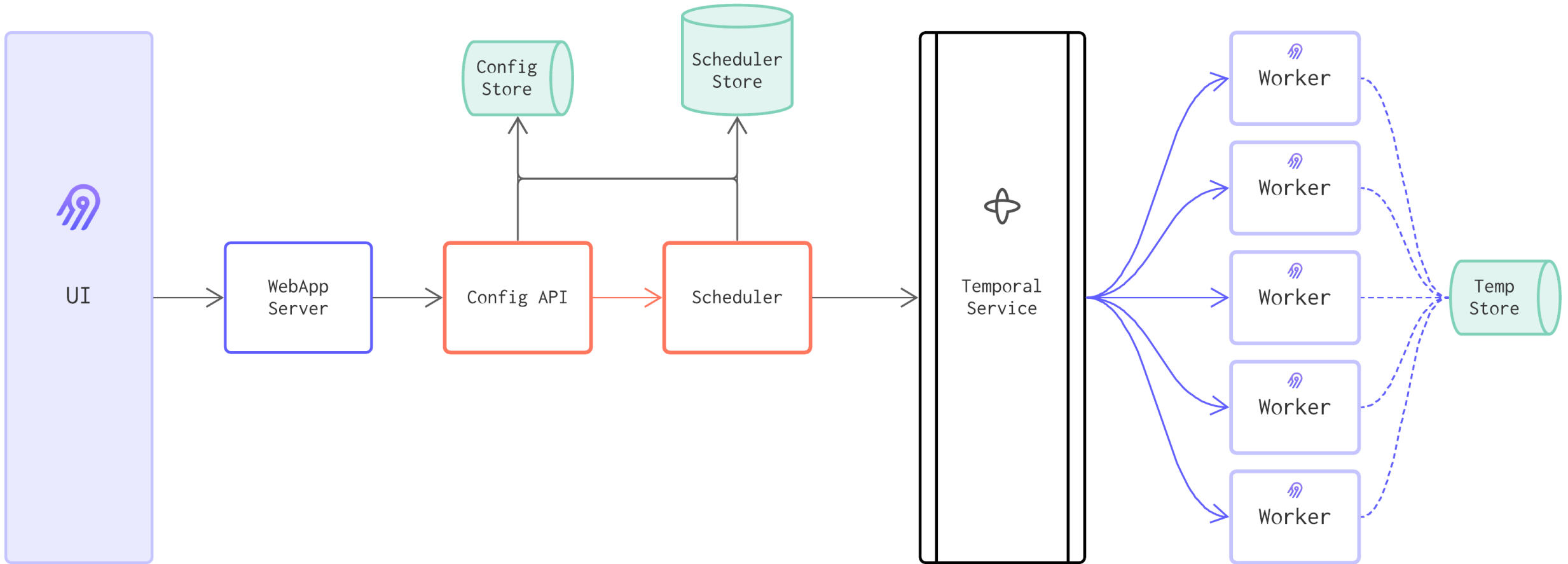


# Pipe and Filter: Ejemplos

- Pipes en Linux
- Pipes de procesamiento de objetos (e.g. machine learning en imagen)
- Procesamiento de audio
- ETLs

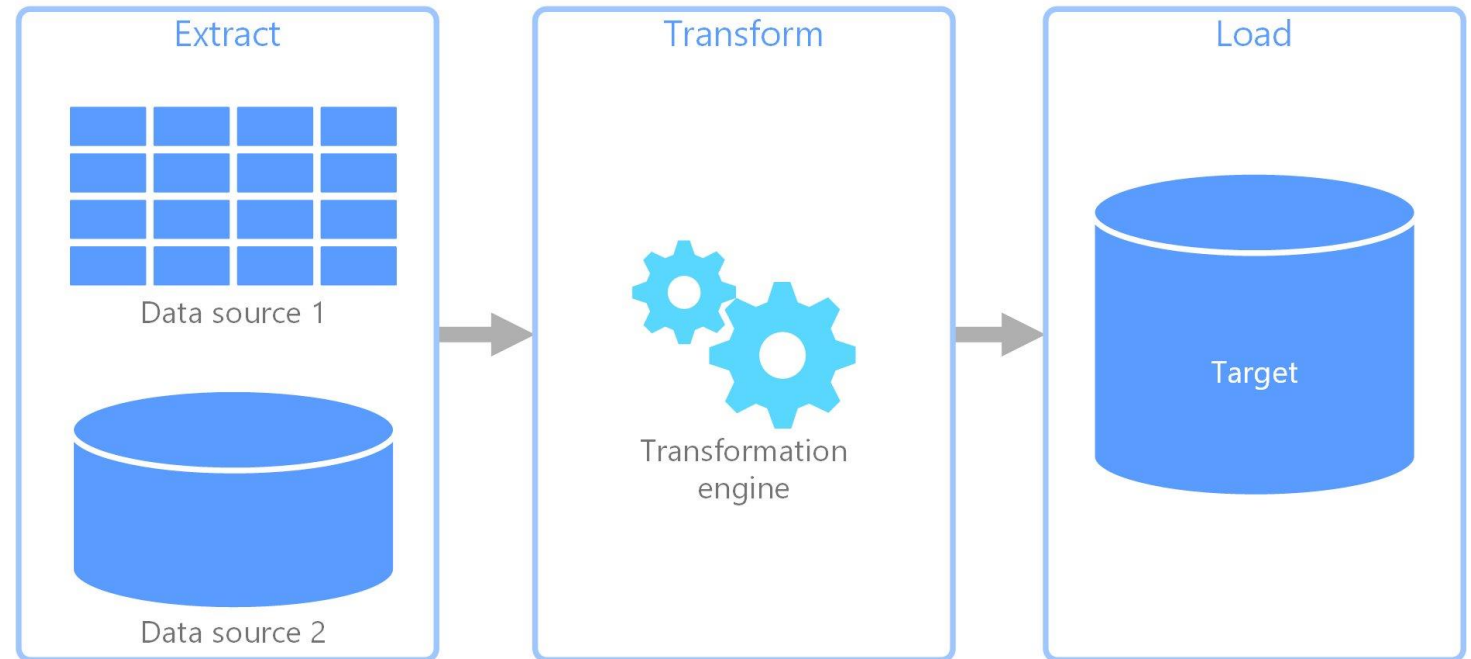






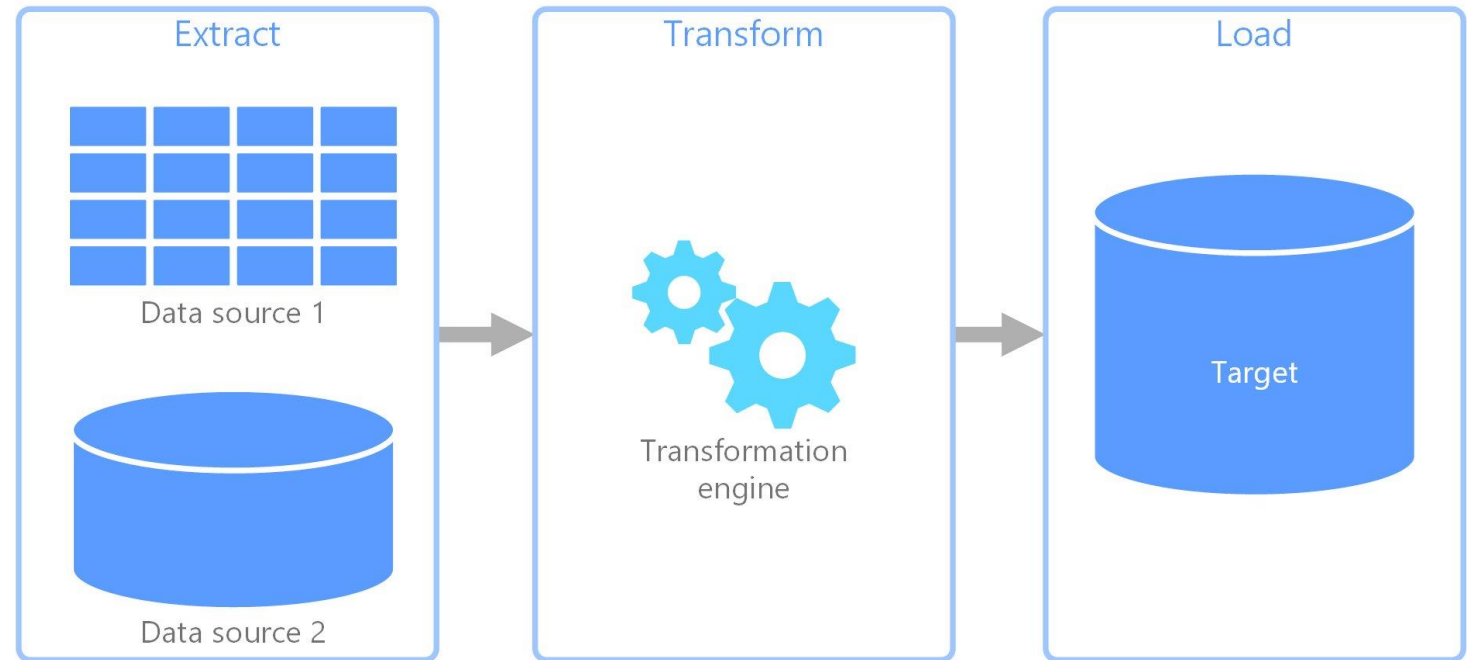
## Pipe and Filter: Pros

- Modular
  - Fácil de modificar módulos y mantener
- Reusable
- Cada componente puede ser escalable
- Flexible
- Testeabilidad



# Pipe and Filter: Cons

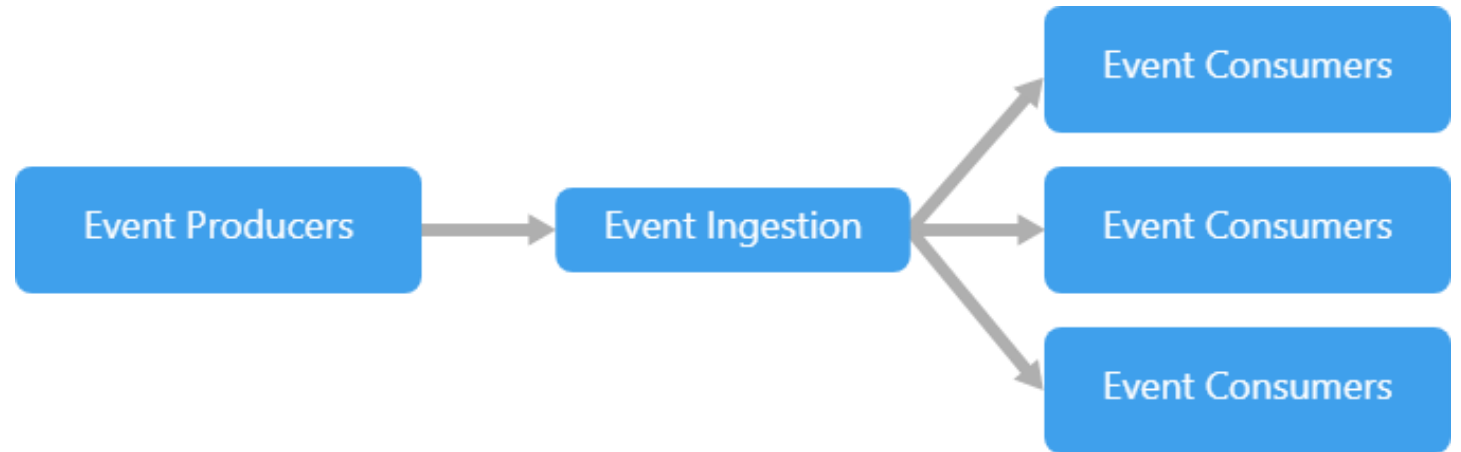
- Requiere un **proceso organizado**
  - No soporta aplicaciones interactivas
- Complejidad
- A medida que el sistema se extiende, **se acopla más**
- No toda la data es compatible
- Se puede acumular *overhead*
- Mínimo común denominador en la transmisión de datos





# Invocación implícita

---



# Invocación implícita: Event Driven Architecture

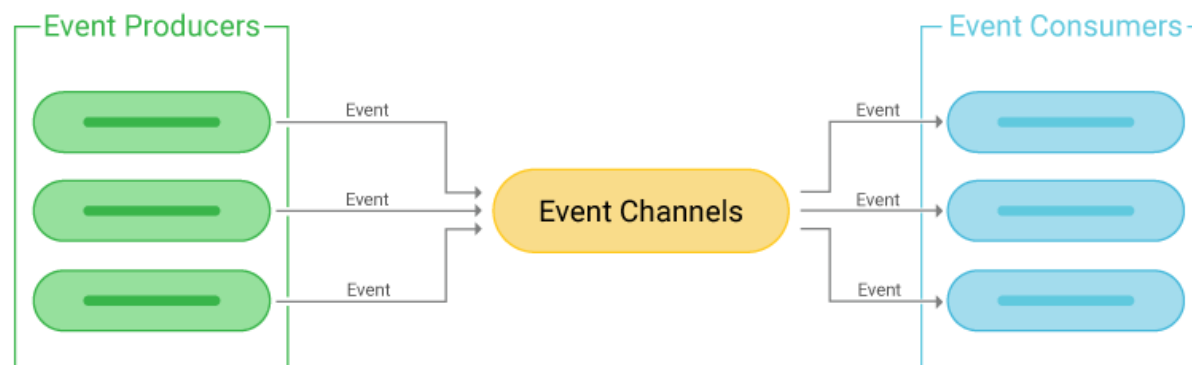
---

- Anuncio de **eventos** en lugar de invocación de métodos
  - **Asincronía** implícita
- **Reacciona a una situación** en particular y toma acción en base a al evento
  - Ejemplo: Enviar una oferta en una subasta online



# Invocación implícita: Event Driven Architecture

- Componentes clave
  - “**Listeners**” registran el interés en algún componente y asocian métodos a eventos (**subscribers, consumers**)
  - “**Producers**” producen contenido (**publishers**)
  - Canales transmiten información entre productores y consumidores
  - Los métodos se **registran de manera implícita**
  - Interfaces de componentes son métodos y eventos
- Conectores
  - Invocación explícita e implícita en respuesta a eventos



# EDA: Características

- Invariantes del estilo
  - “Publicadores” no conocen los efectos de los eventos
  - No se asume cómo se procesará la respuesta a eventos
  - No se responde a todo necesariamente
  - “Fire and forget”



# EDA: Características

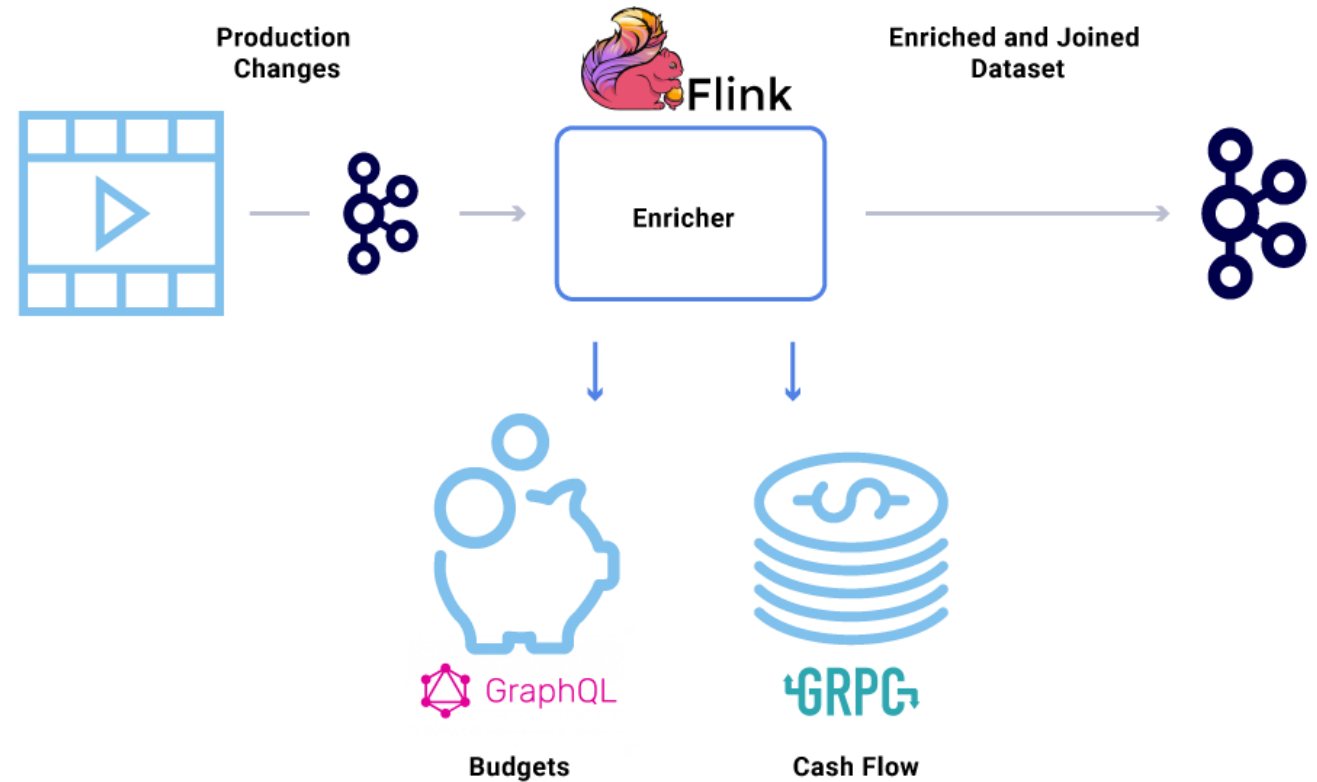
---

- Datos
  - Suscripciones
  - Notificaciones
    - Push & Pull
  - Elementos transferidos
    - Según contrato
    - Llevan toda la data necesaria
- Topología
  - Red
  - Broker
  - Mediador



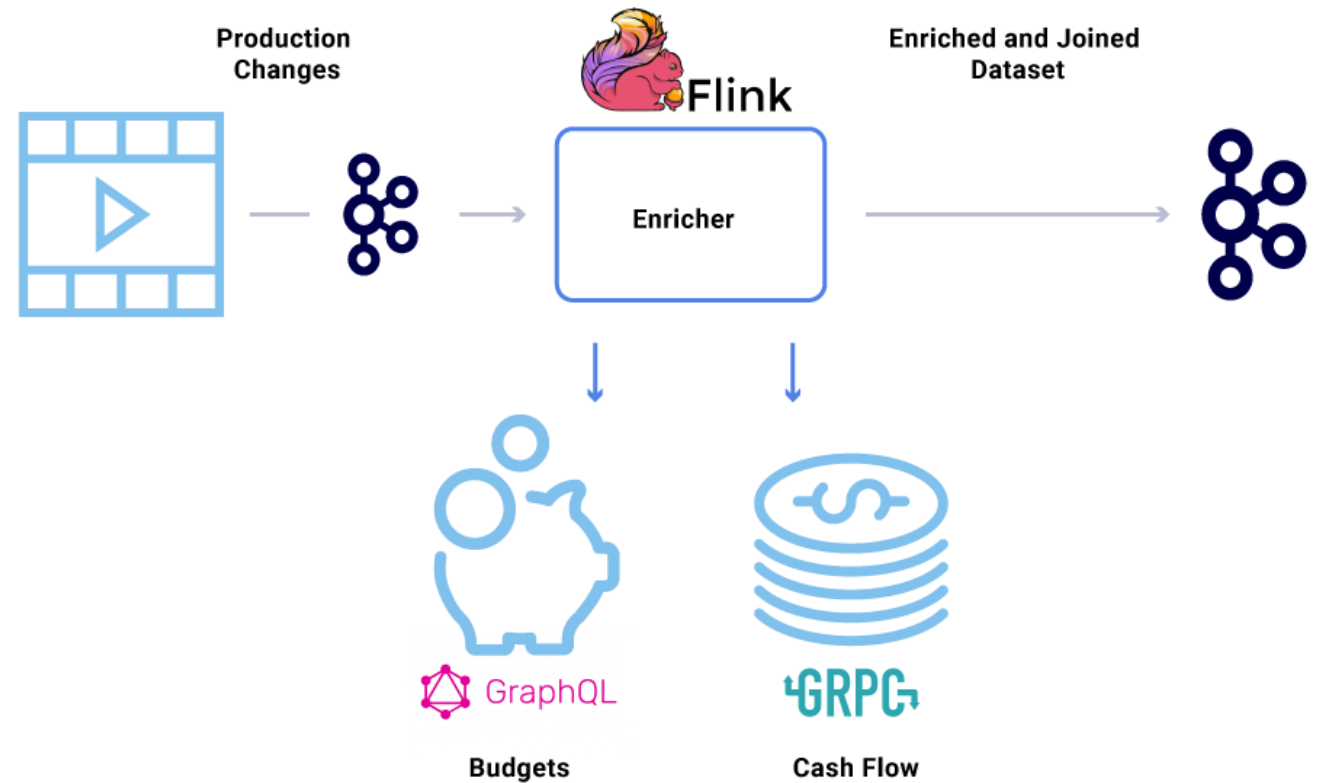
# EDA: Pros

- **Reuso de componentes**, evolución del sistema
  - Puede pasar de una app pequeña a un gran sistema
- **Reducción de acoplamiento**
  - Más compatible con agilidad
  - Flexible
  - Modularizable
  - Compatible con microservicios
- Manejo elevado de **escalabilidad y distribución**
- Alto **performance**



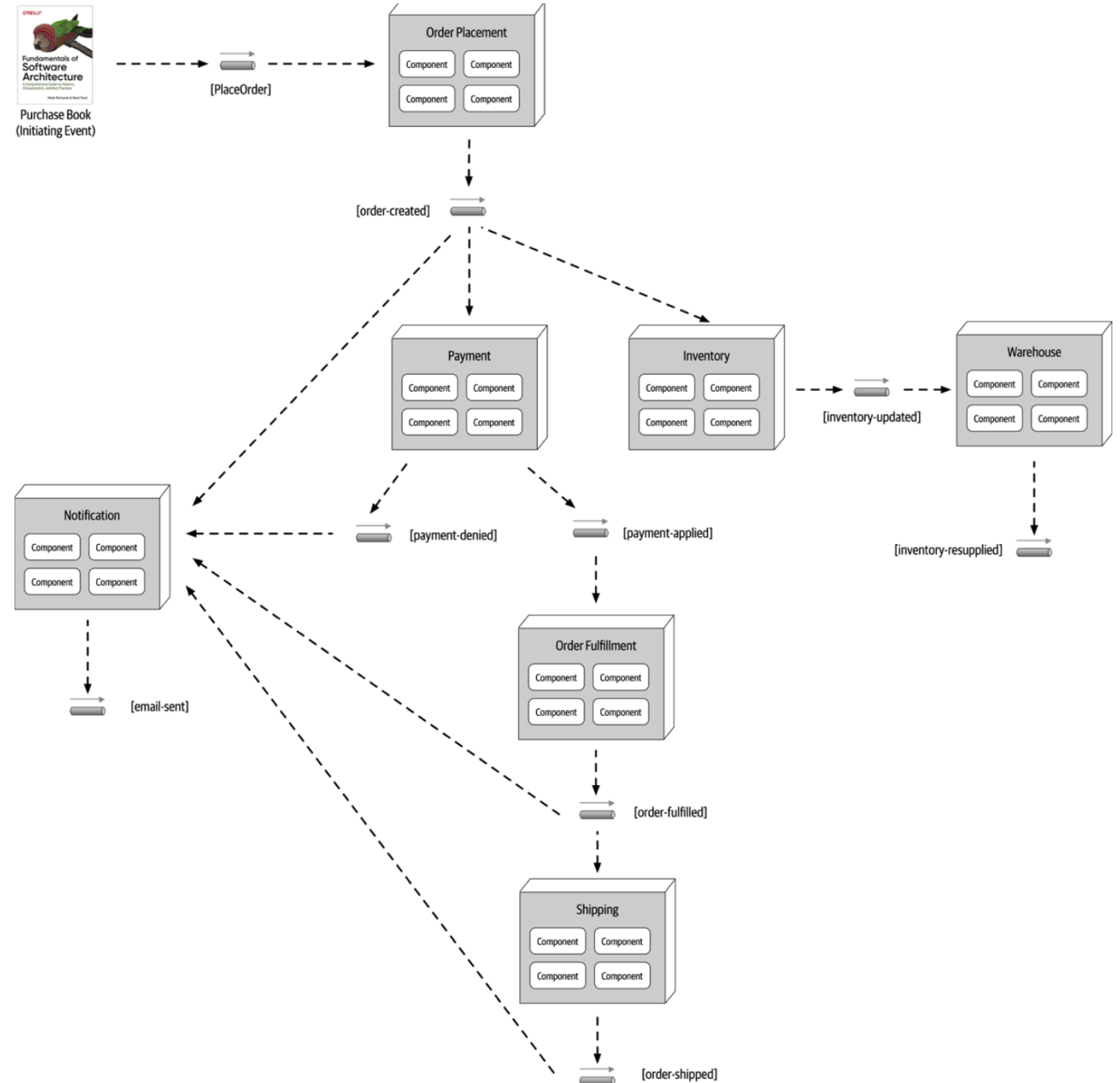
## EDA: Pros

- No usa integración punto a punto: **agregar consumidores es muy sencillo** en cantidad y capacidad
- Muy compatible con tiempo real
- Es posible flexibilizar y añadir *request reply*
  - Incluso añadir colas y flexibilizar capacidad de procesamiento



# EDA: Cons

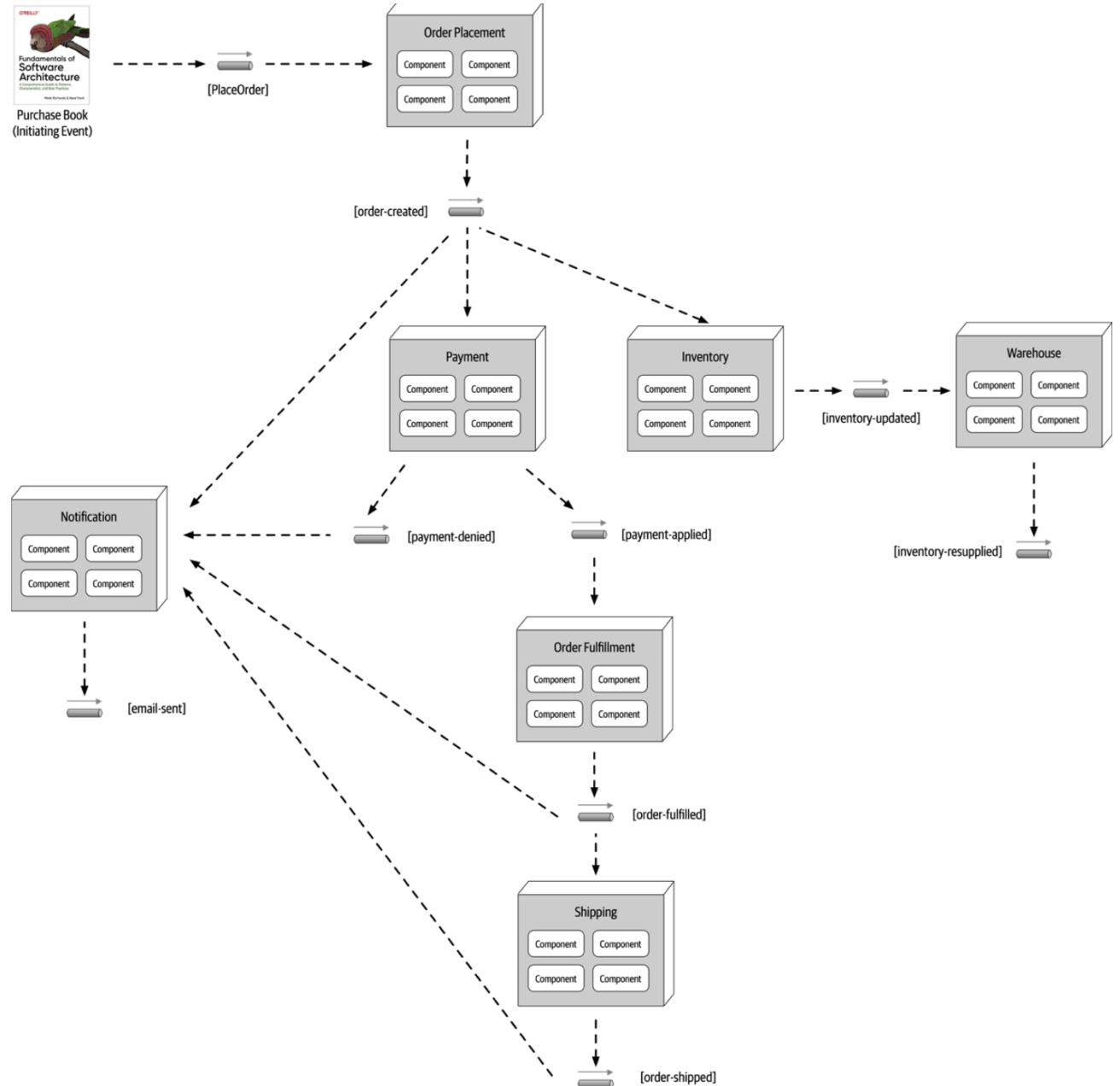
- Estructura de sistema **no es intuitiva**
  - Se eleva la complejidad
  - ¿Quién habla a quién?
  - Asincronía
- Latencia
  - **Eventos complejos pueden elevar latencia**
- Eventualmente consistente
  - Olvidarse de ACID
- Soporte y **mantenibilidad**
  - *Debugging*
  - Búsqueda de problemas





# EDA: Cons

- No hay conocimiento de la **respuesta de los componentes al evento**, ni del orden de las respuestas
  - Garantía de envío no asegurada
  - E.g. IoT
- Componentes **no permiten control del sistema**
  - Se necesitarían intermediarios caros
  - Difícil de restaurar la operación





## EDA: Consideraciones

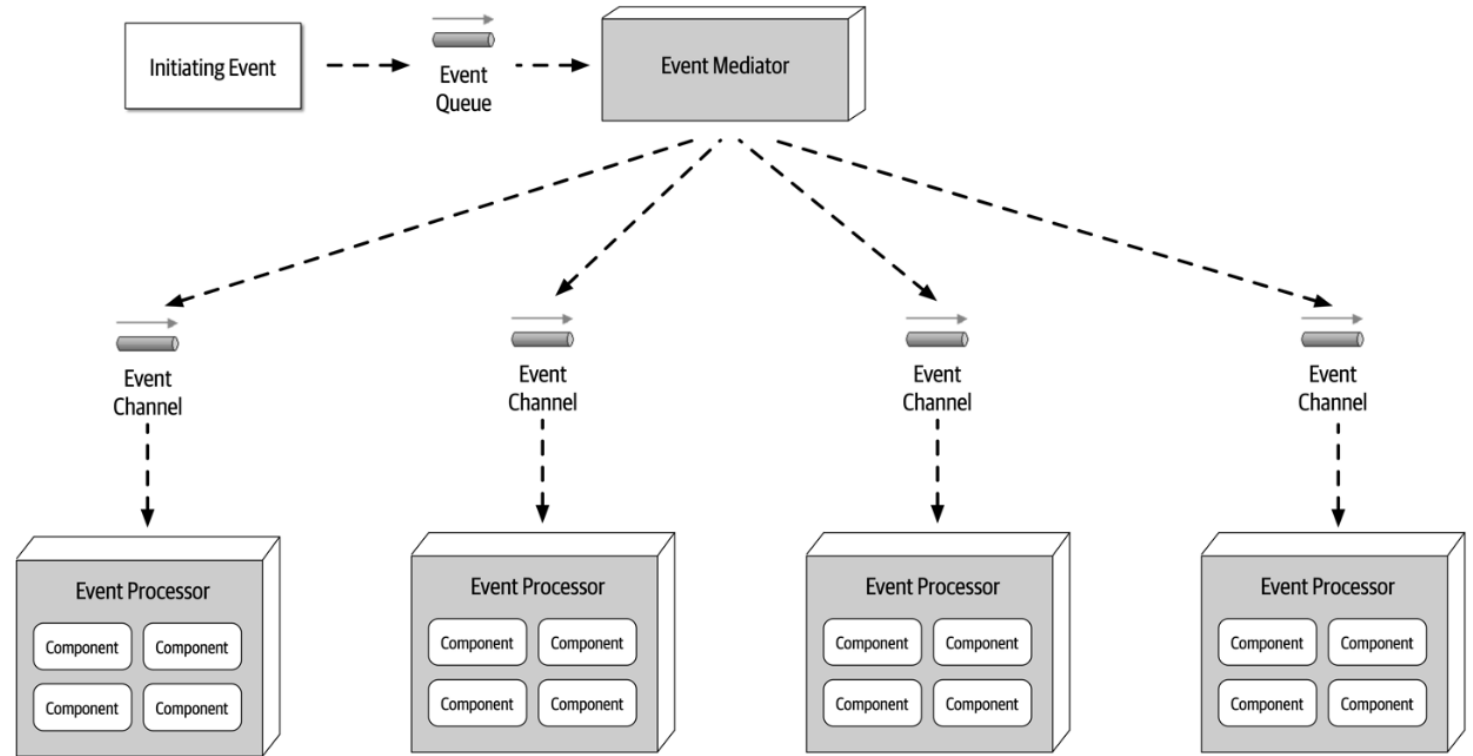
---

- Establecer **buenos contratos**
- **Monitorear** eventos lo más posible
- Idempotencia: **Manejo de eventos duplicados/desordenados**
- Manejar errores: Responder y notificar



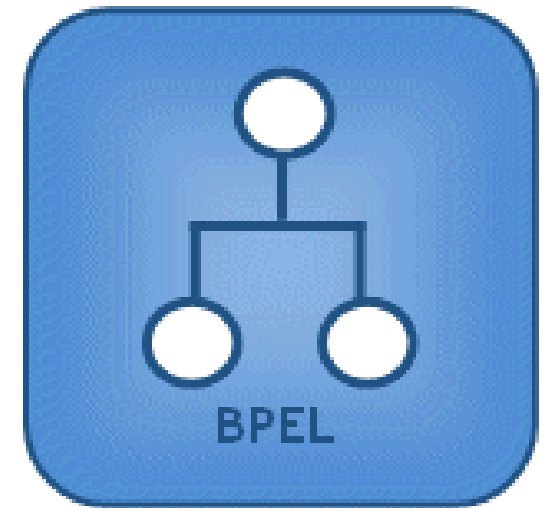
# EDA - Topologías: Mediator

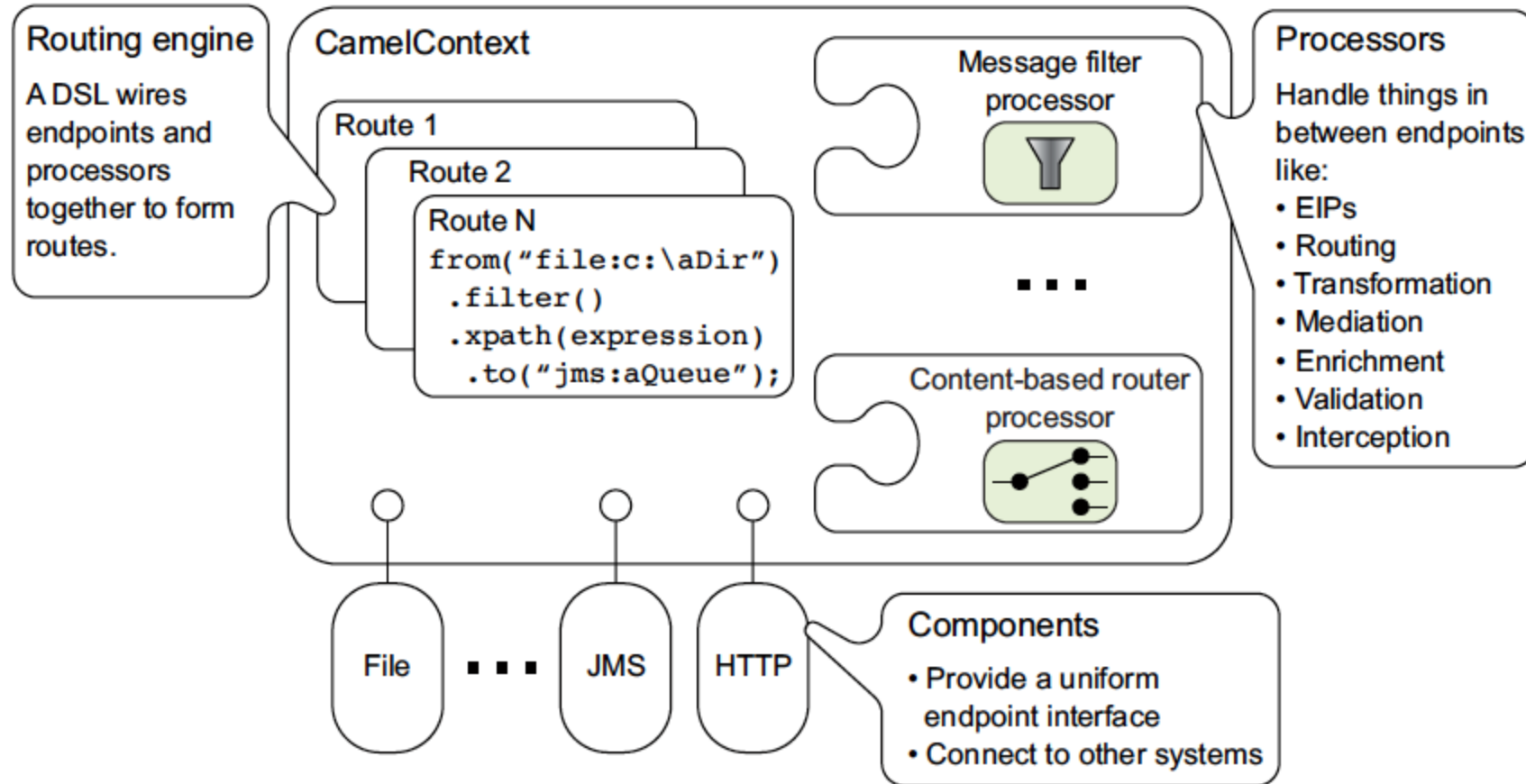
- Ente centralizado capaz de movilizar un proceso dado
- Toma los eventos y los mueve entre procesadores de eventos
- Cuando necesitamos **control del flujo**
- Puede tener comportamiento **síncrono**



## EDA - Topologías: Mediator

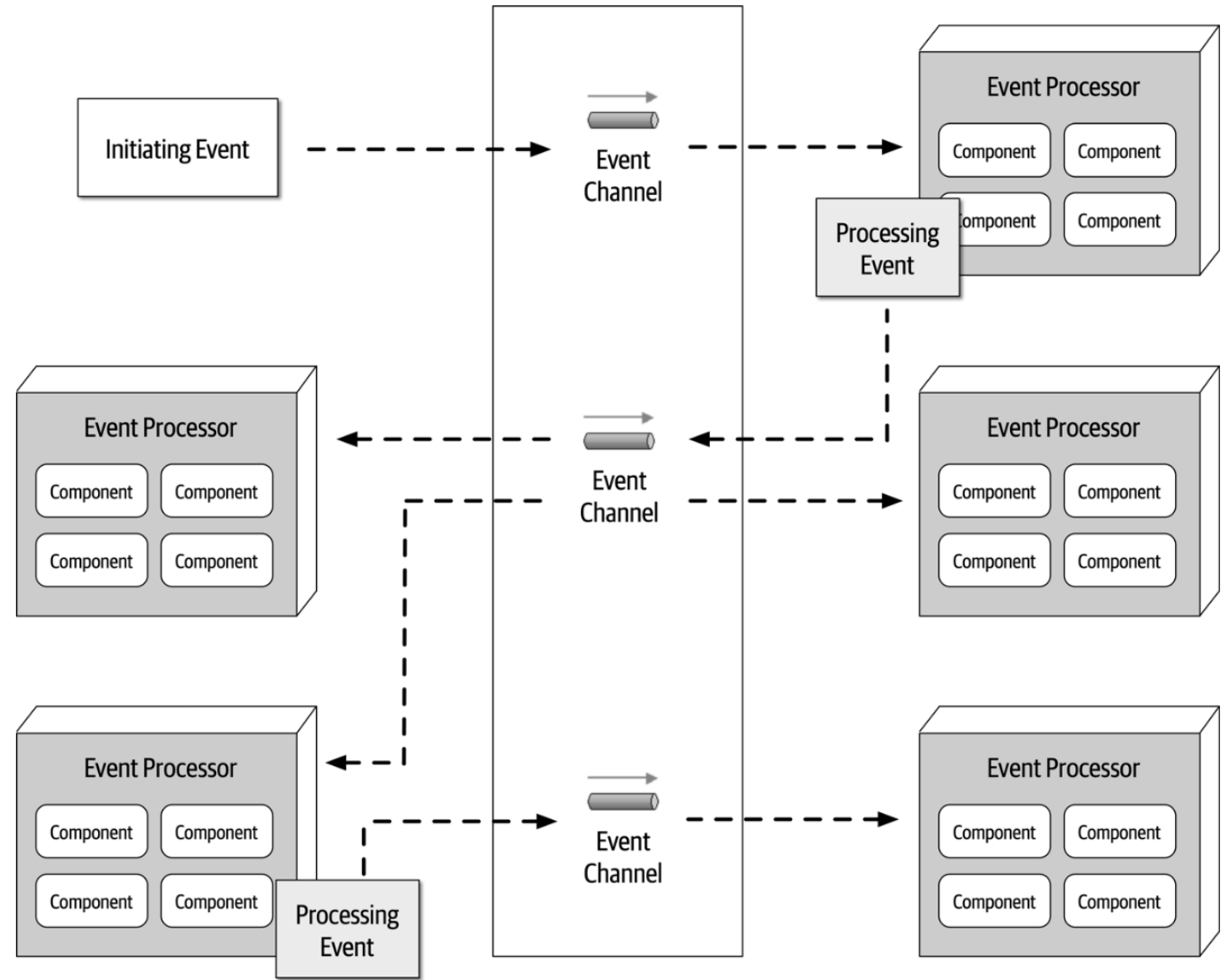
- Ejemplos:
  - Apache Camel
  - Oracle BPEL
  - Django signals





# EDA - Topologías: Broker

- Entidad dedicada a **enviar mensajes y cumplir entregas**
- No orquesta eventos, **solo los mueve**
- Intermediario necesario para sustentar muchas arquitecturas de eventos
  - Ojo con SPOF
- Puede usar un protocolo (AMQP, XMPP)

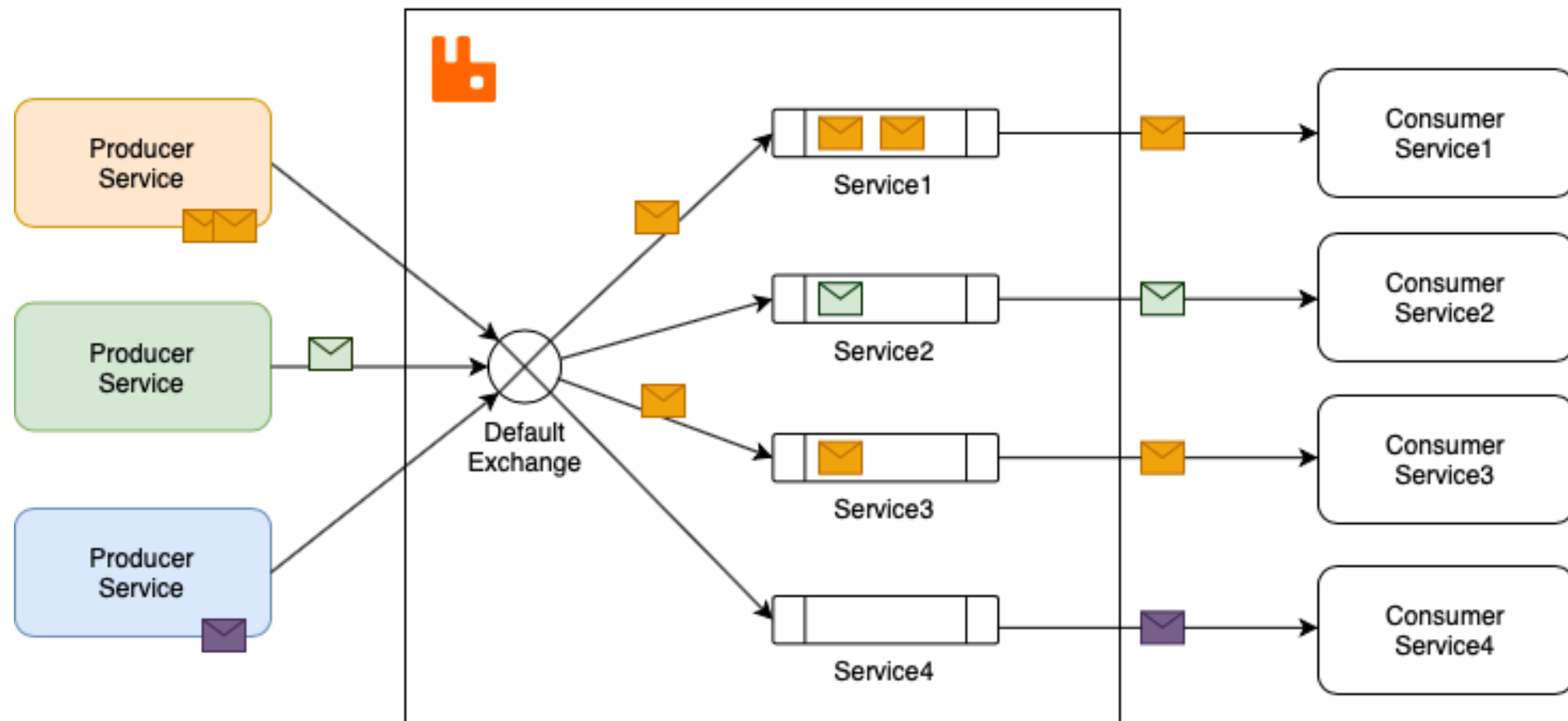


## EDA - Topologías: Broker

- Ejemplos:
  - RabbitMQ
  - Mosquitto
  - Redis
  - Ejabberd
  - AWS SNS

The RabbitMQ logo features an orange icon of a rabbit head in profile, facing right, followed by the text "RabbitMQ" in a sans-serif font. "Rabbit" is orange and "MQ" is grey.The Ejabberd logo is written in a blue, rounded, lowercase font. The letter "e" at the end is stylized, resembling a speech bubble or a drop.

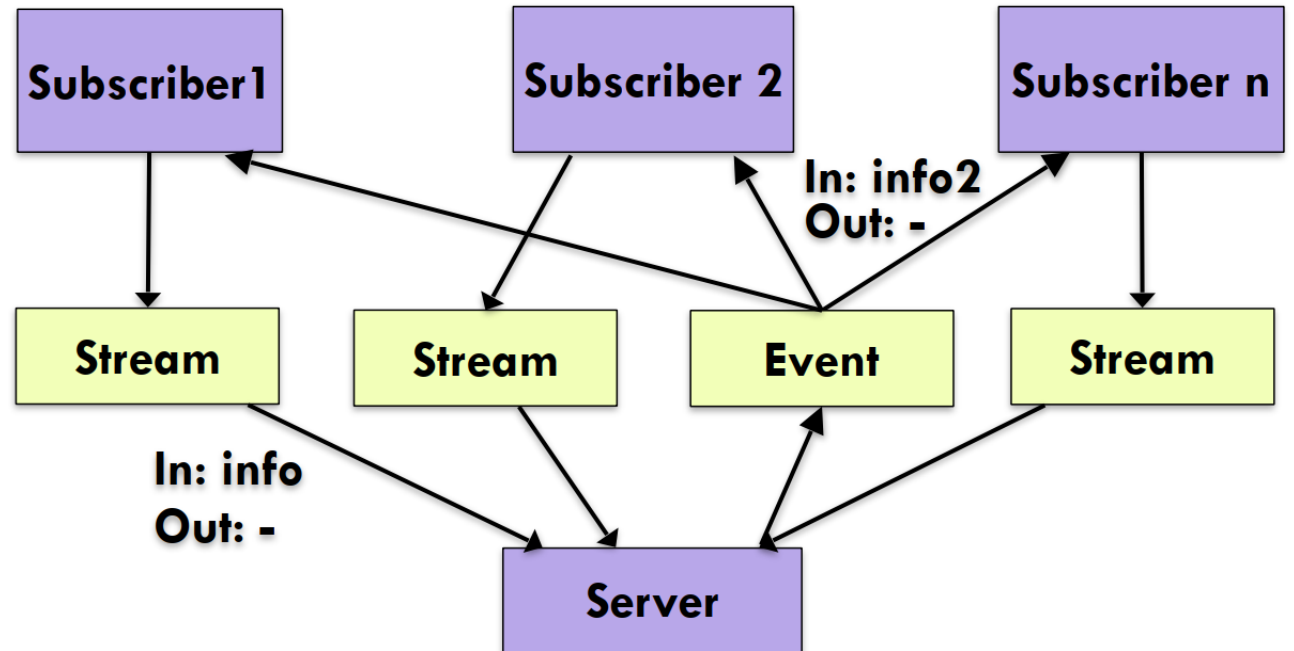
# RabbitMQ





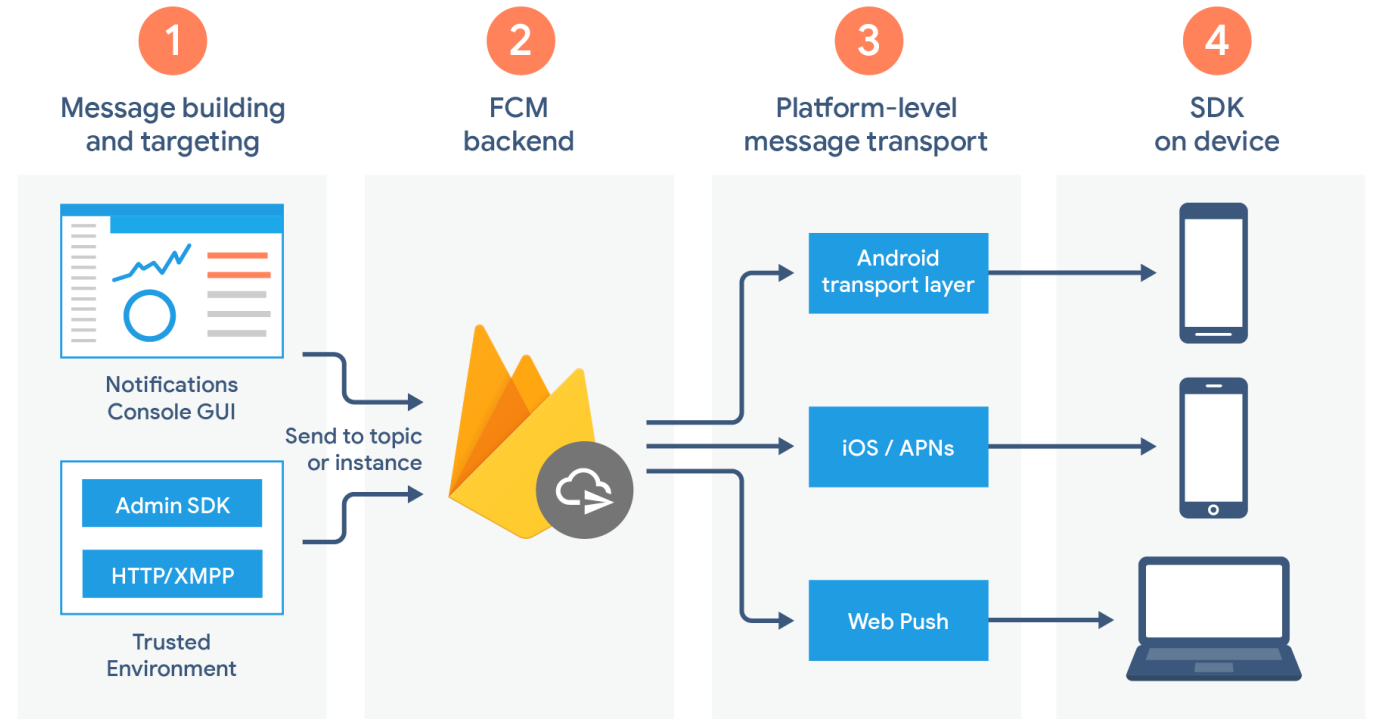
# Variante de EDA: Publisher/Subscriber

- Publishers envían mensajes **sin necesariamente apuntar a suscriptores** específicos
- Suscriptores escuchan mensajes
  - No todos interesan
  - Solo usan la información que les interesa
- Puede **ordenarse en tópicos** para mejorar separación
- Mensajes se envían de forma "indirecta"

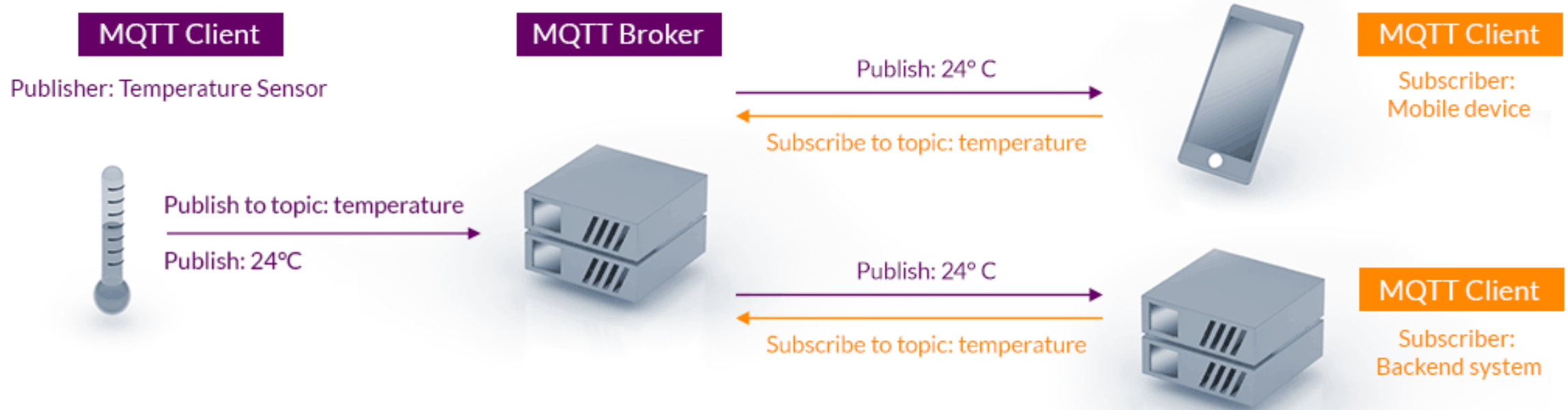


# Pub/Sub: Ejemplos

- Publishers
  - RSS
- Chats & gaming
  - XMPP
- IoT
  - Protocolo MQTT
- E-commerce
  - Eventos críticos
  - Stock
- Envío de correos

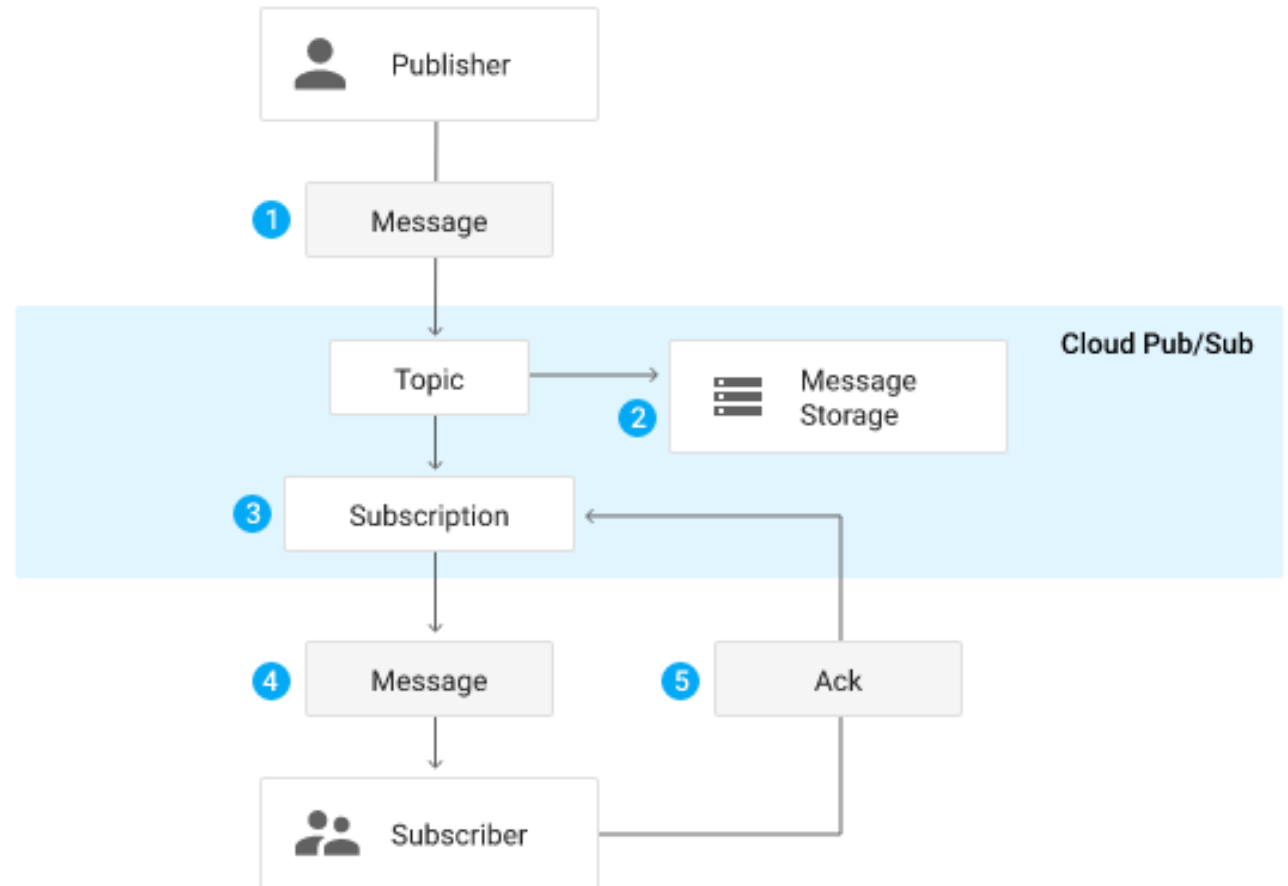


# MQTT



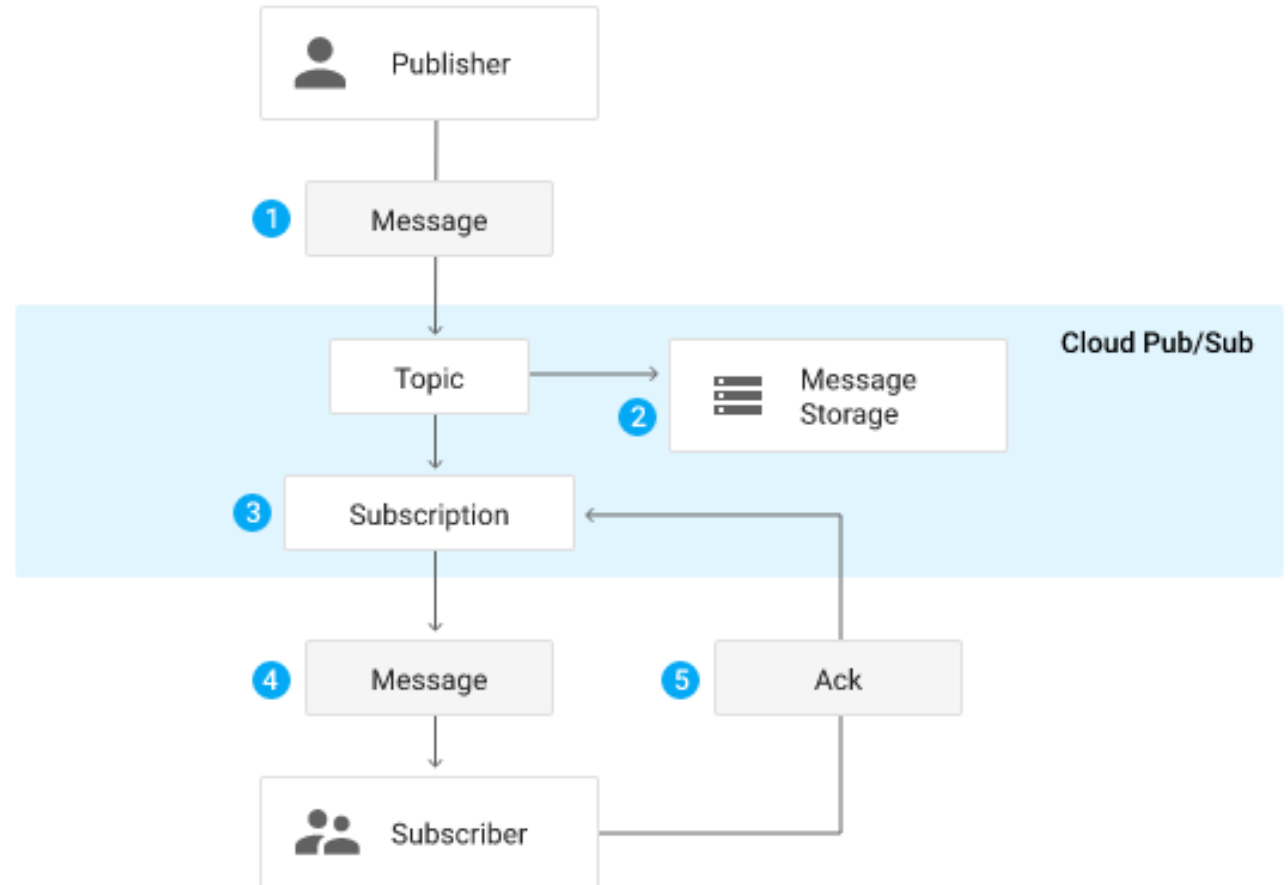
# Pub/Sub: Pros

- Ventajas de EDA
- Desacoplamiento
  - Cualquiera podría entrar a un tópico
- Asincronía
  - *Fire-and-Forget*
  - Podrías esperar un evento
- Escalable
  - Aprovechamos *broadcasting*



# Pub/Sub: Cons

- Mensajería confiable
  - ¿Cómo me aseguro de que llegue?
  - ¿Realmente los *brokers* son tan confiables?
  - QoS
- Seguridad
  - Control de acceso (RBAC, CBAC)
- Complejidad
  - Podrían enredarse muchos tópicos y contenidos

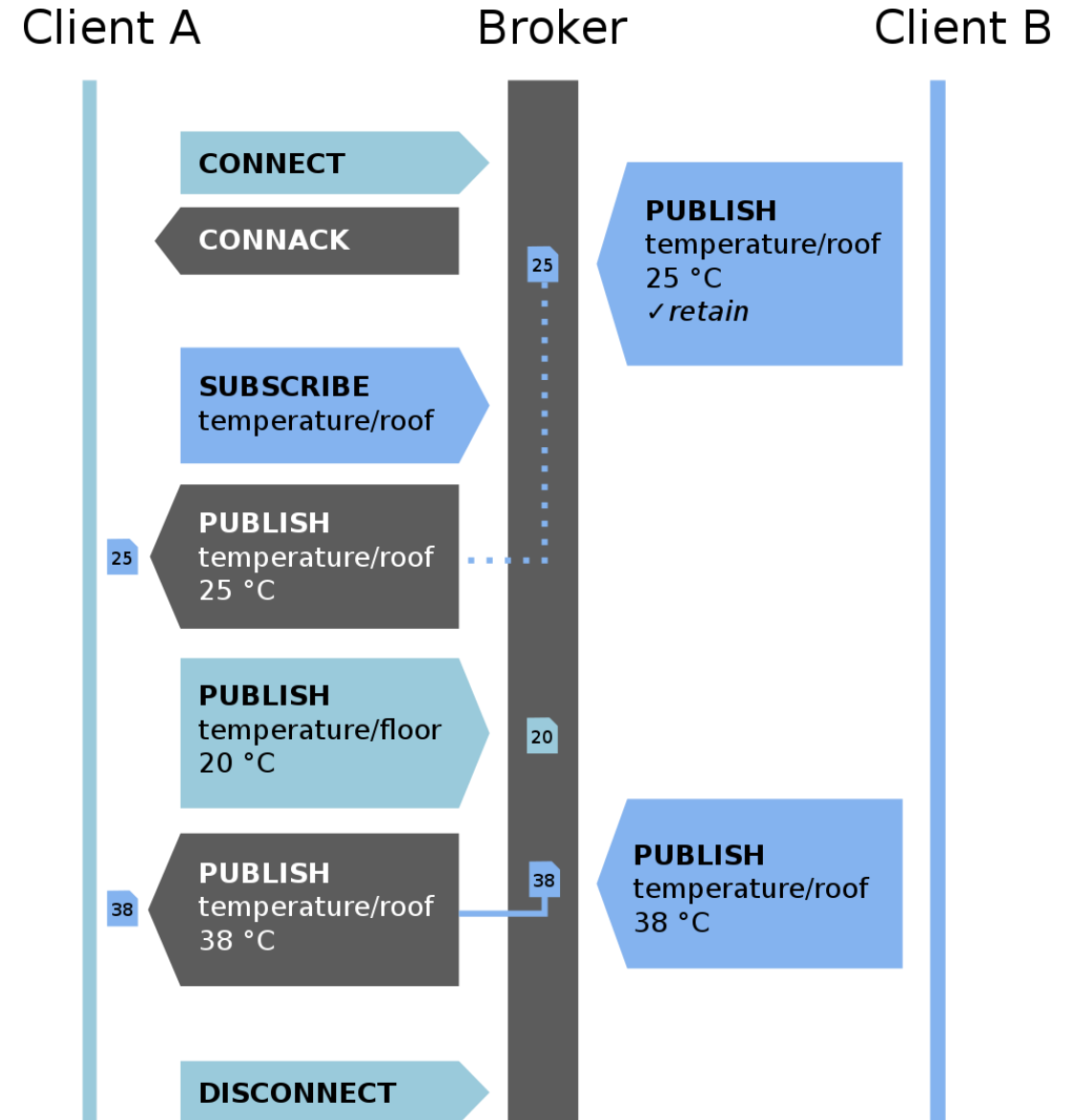


# Pub/sub: Otros ejemplos

| Criterio               | XML-RPC ping   | changes.xml    | SUP            | SLAP           | XMPP pubsub | rssCloud          | PubSubH ubbub |
|------------------------|----------------|----------------|----------------|----------------|-------------|-------------------|---------------|
| Transporte             | HTTP           | HTTP           | HTTP/HTTPS     | UDP            | TCP/XMPP    | HTTP              | HTTP/HTTPS    |
| Estilo distribución    | Ping/Poll      | Polling        | Polling        | Ping/Poll      | Push        | Ping/Poll         | Push          |
| Latencia               | Bajo           | Alto           | Alto           | Bajo           | Minimo      | Bajo              | Minimo        |
| Thundering herd        | Si             | Si             | Si             | Si             | No          | Si                | No            |
| Spamable               | Si             | Si             | No             | No             | No          | No                | No            |
| DoS Publishers         | Prevenible     | No             | No             | Prevenible     | Prevenible  | Prevenible        | Prevenible    |
| DoS Relay              | Si             | No             | No             | No             | No          | Si                | No            |
| Hosting barato         | No             | No             | No             | No             | No          | Maybe             | Yes           |
| Formato msjes          | XML schema     | XML schema     | JSON           | Binary packet  | XMPP        | XML schema        | RSS o Atom    |
| Notificaciones seguras | No             | No             | Algo           | No             | Si          | No                | Si            |
| Complejidad publisher  | XML-RPC client | XML-RPC client | SUP IDs        | UDP send       | XMPP send   | XML-RPC/REST ping | REST ping     |
| Complejidad subscriber | Crawl pipeline | Crawl pipeline | Crawl pipeline | Crawl pipeline | XMPP client | Crawl pipeline    | WebApp        |

# En su proyecto

- Fácil agregar *publishers* y *subscribers*.
  - Tenemos 152 suscriptores y podrían ser más
  - Ahora pueden ser publicadores en canales permitidos
- MQTT: *MQ Telemetry Transport*
  - Fácil de escalar
  - Mínima configuración
  - Conexión simple y ligera, soporta QoS
  - Usado en IoT
  - Corre sobre TCP/IP



# Métodos de identificación de componentes

---

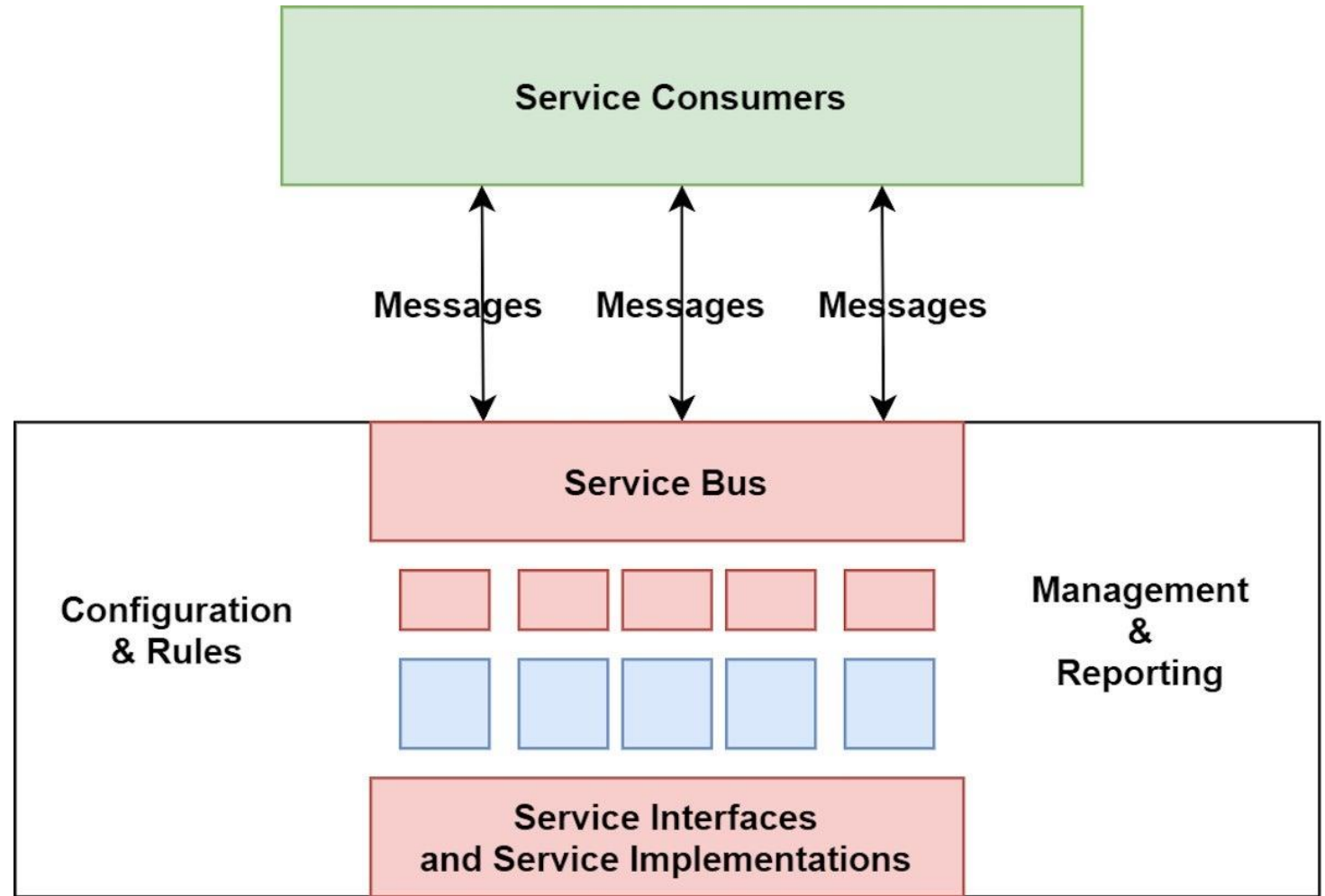
- Modelo actor/acción (Desde RUP)
  - Usuario hace X sobre Y con rol Z
  - Ayuda a capturar requisitos
  - Muy cercano a historias de usuario
- *Event storming*
  - ¿Qué eventos gobiernan la app?
  - Ejemplo en proyecto
  - Buen alcance de complejidad
- Partición dominio vs técnico:
  - Ejemplo de decisión: En capas vs Servicios
  - Usualmente el que conocemos solo es el técnico



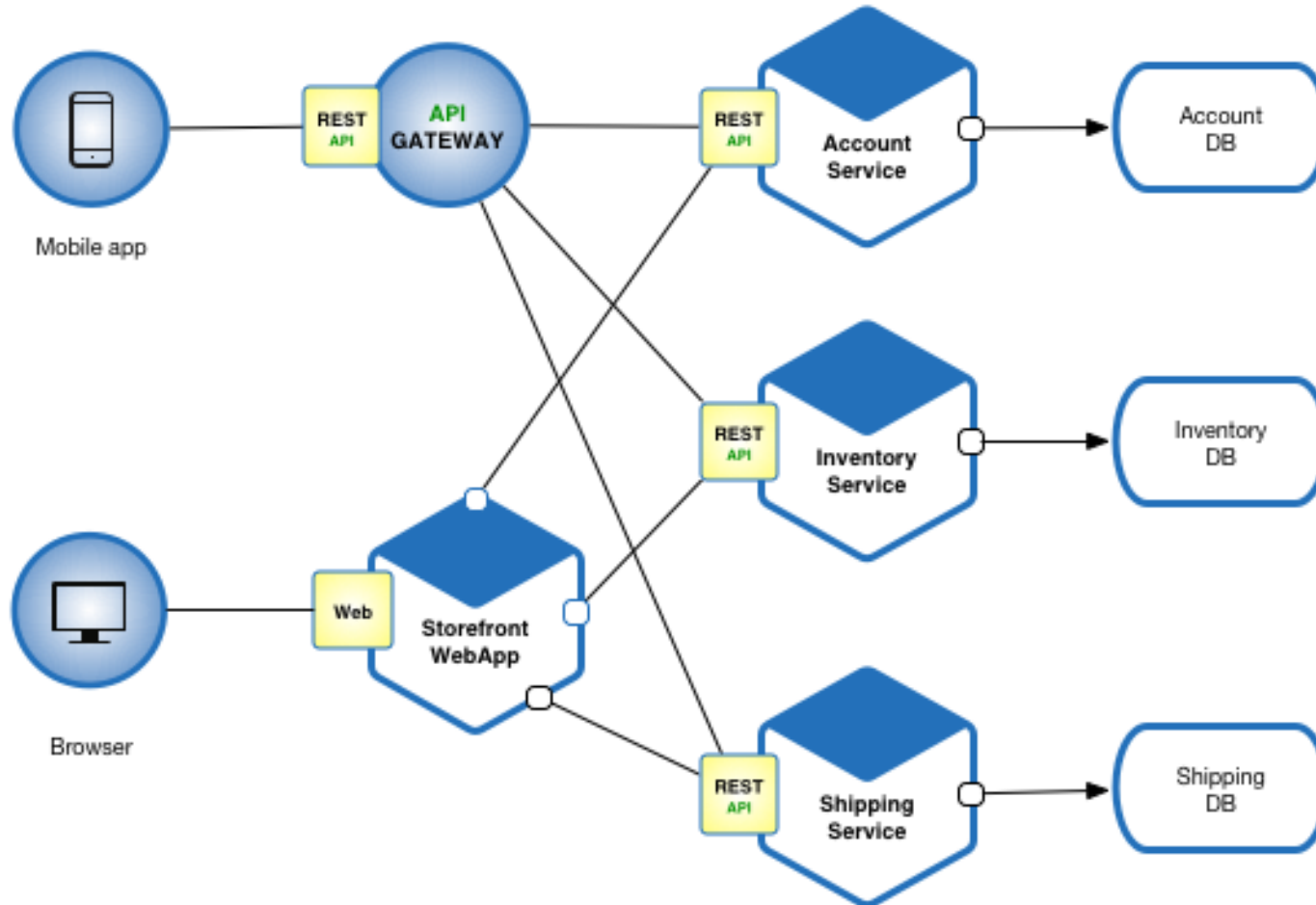


# Service oriented Architecture

- Paradigma que usa el **servicio** como idea básica para permitir el desarrollo de aplicaciones **interoperables**, masivamente distribuidas, capaces de **evolucionar**, de manera rápida y a bajo costo
- Servicios de grano grueso

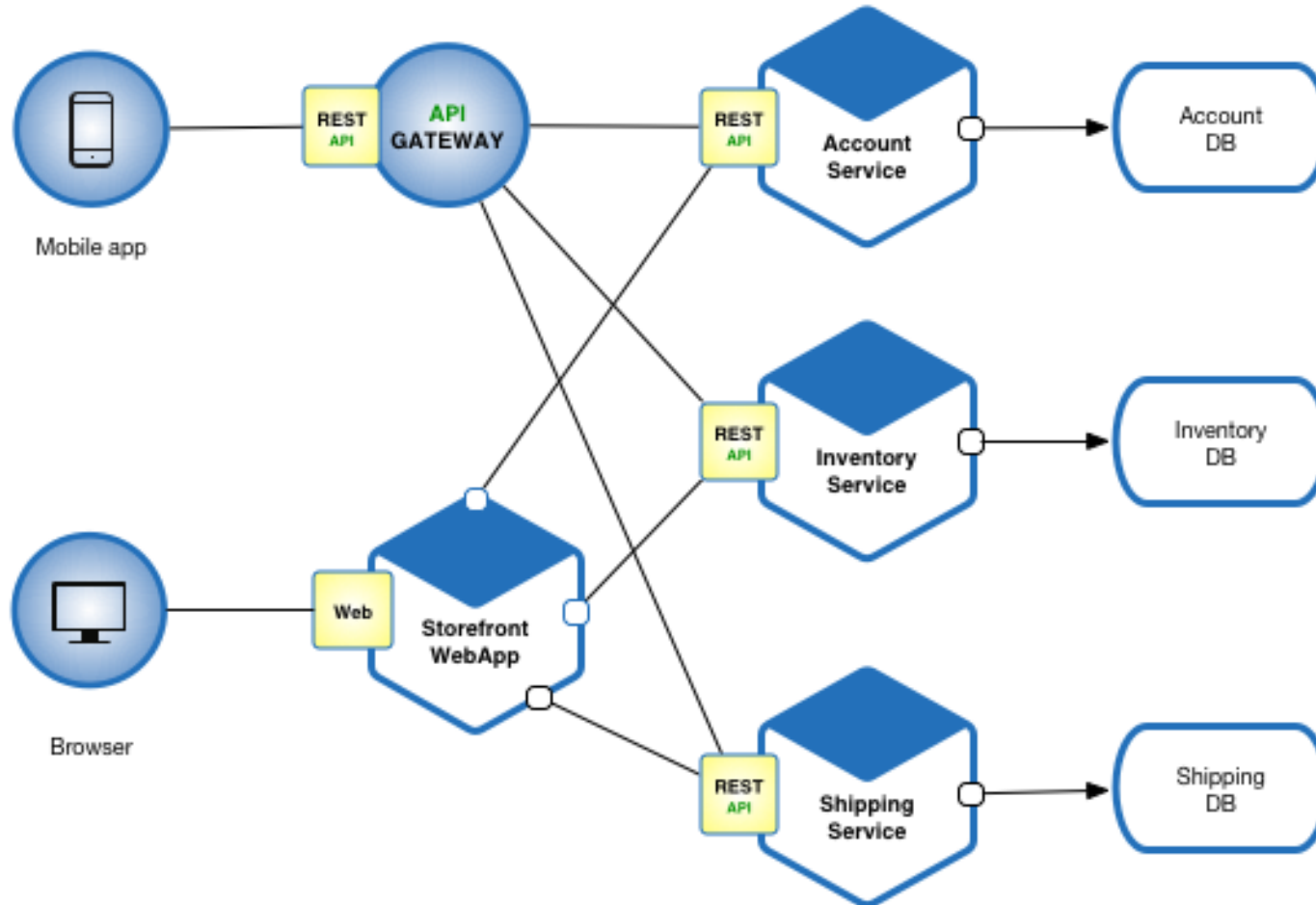


# Microservicios



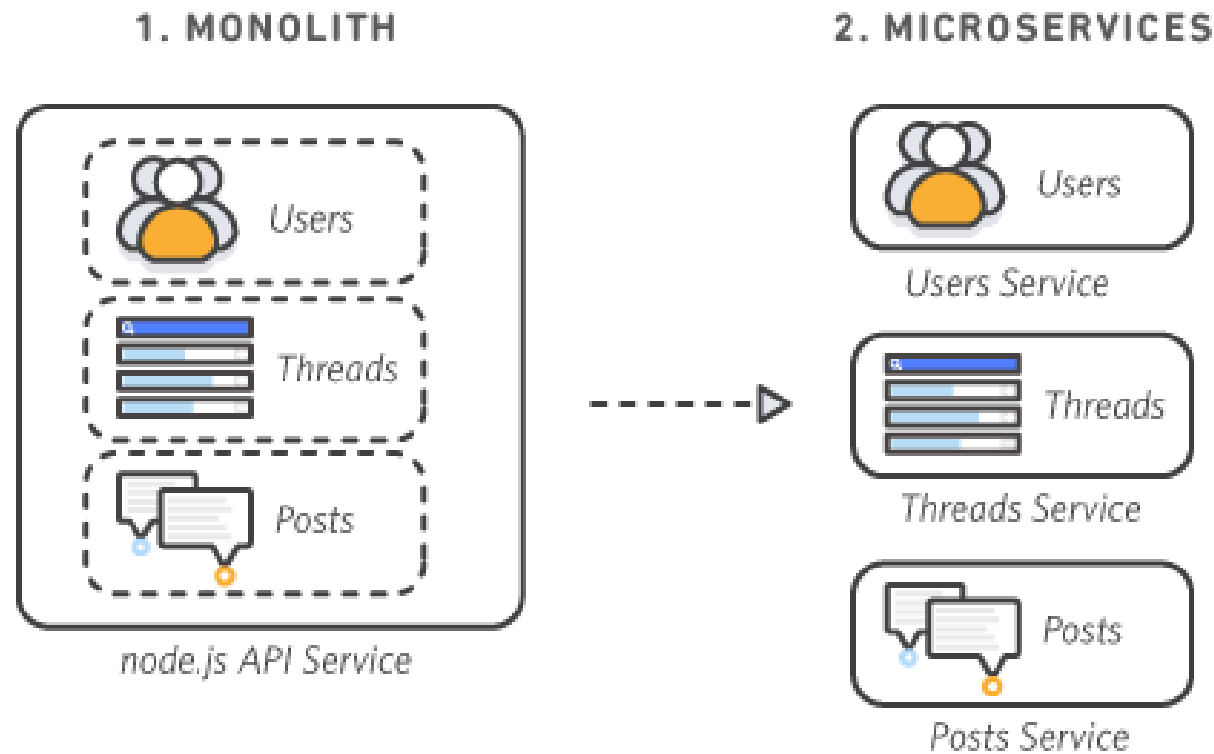
- Variante de SoA
  - Ligera en implementación y costos
- Servicios débilmente acoplados que cumplen una función
  - Determinado por DDD u otro método
  - Suele encapsular una funcionalidad de negocio o dominio
  - Grano fino

# Microservicios



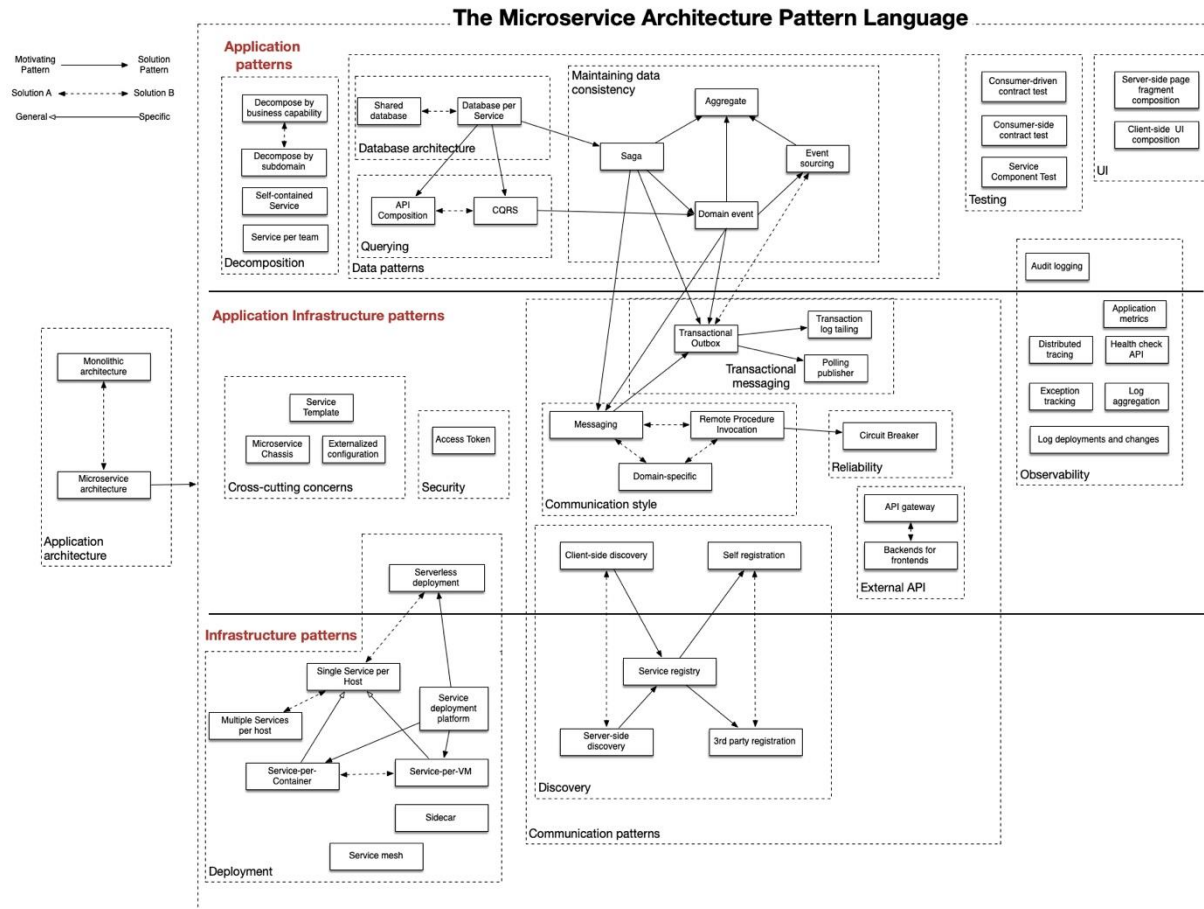
- Independientemente desplegables
  - CI/CD esencial
- Dominio sobre su fuente de datos
- Otros perks
  - Independencia de código
  - Contención de seguridad
  - DevEx mejorada
  - Evolutivo

# Principios clave en microservicios



- Domain Driven design
- Cultura DevSecOps
- Diseño para fallar
- Consumer-first

# Patrones en microservicios



- Saga
- Sidecar
- CQRS
- Circuit breaker
- Anti-corruption layer
- Event sourcing

# Sobre los debates

- Podrán inscribir un grupo en una sección
- Recibirán una especificación 5 días antes de exponer, que podría ser
  - Caso
  - Decisiones
- Expondrán 2 grupos por clase
- Cada semana se volverá más exigente la evaluación
- 1 grupo en exposición, otro de stakeholders
  - Como cliente
  - Como arquitecto
  - Como desarrollador







# Estilos y Patrones IV

---

IIC2173 - Arquitectura de sistemas de software

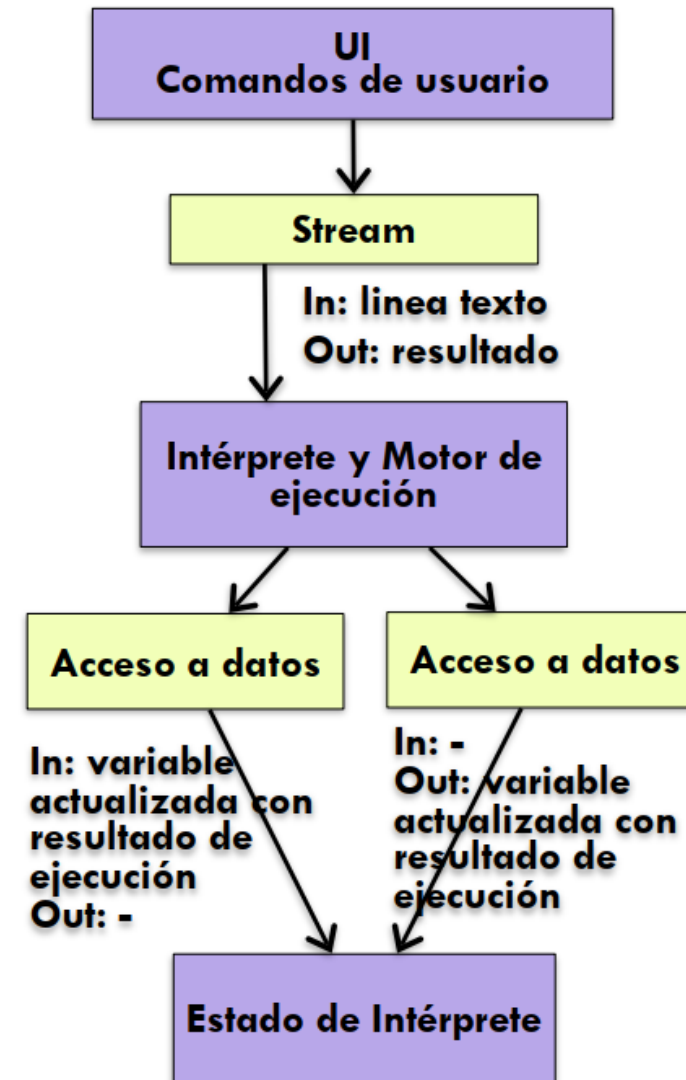
Departamento de Ciencia de la Computación

Escuela de Ingeniería – PUC

Nicolás Acosta – Gerardo Crot

# Intérprete

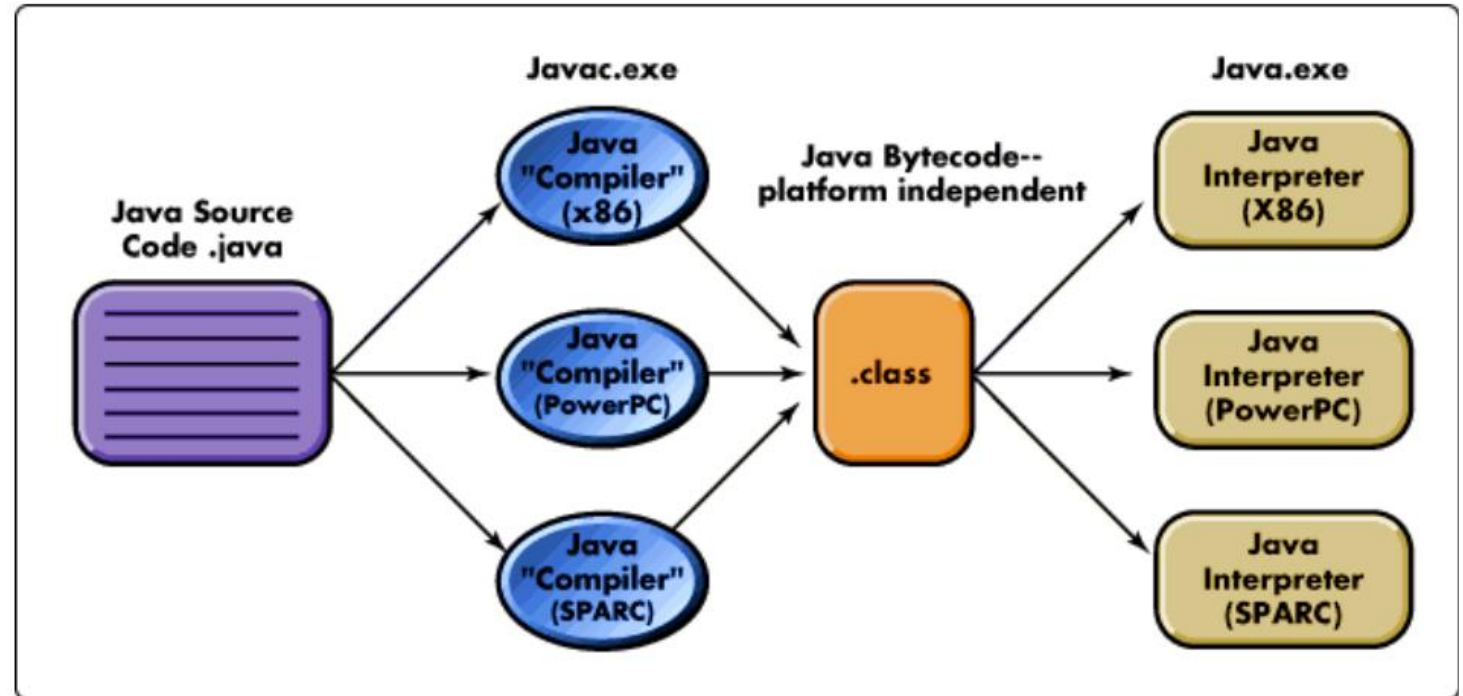
- El intérprete **parsea y ejecuta comandos de entrada**, actualiza el estado mantenido por el intérprete
- Componentes
  - Intérprete de comandos, programa de interpretación del estado, interfaz de usuario





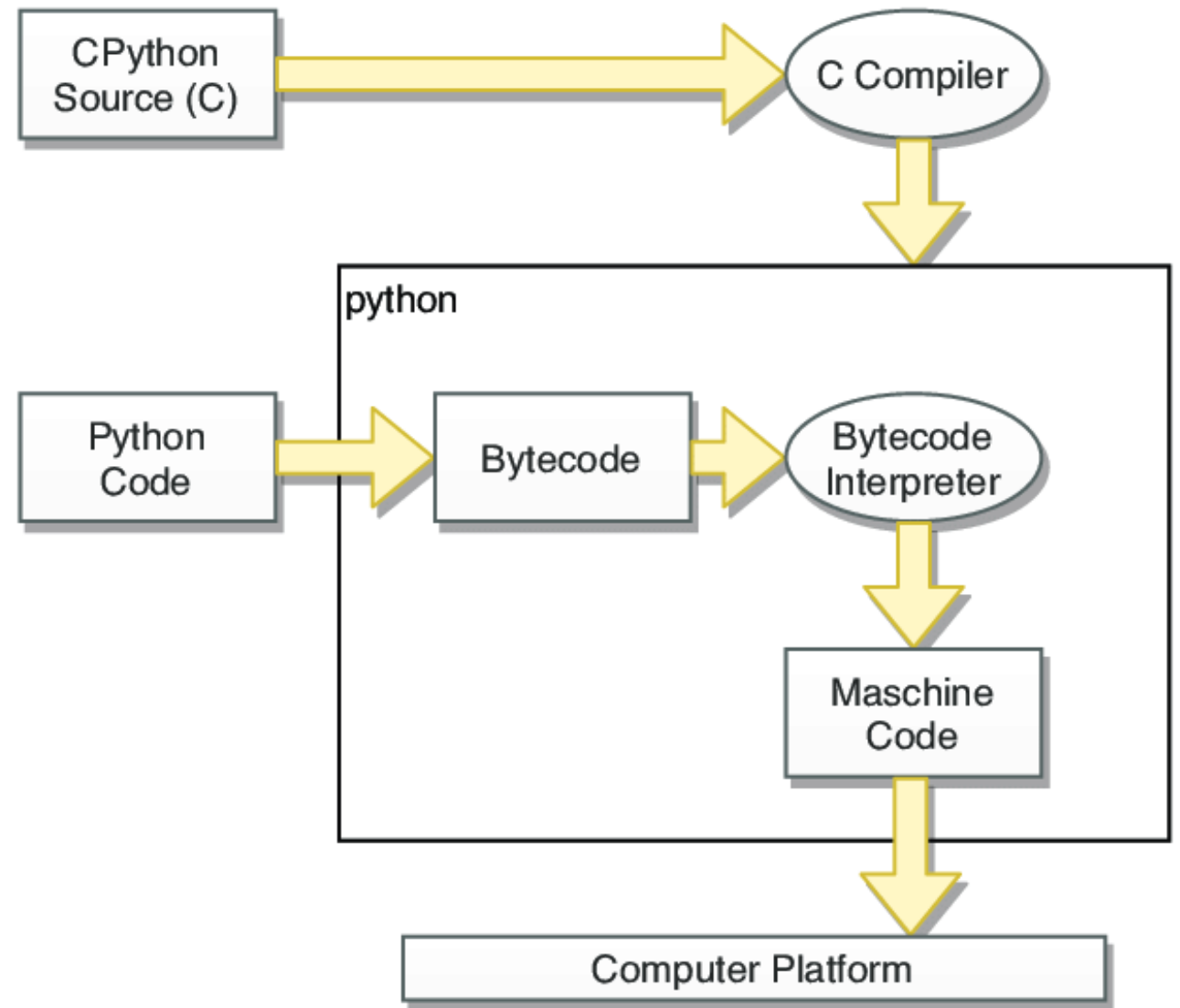
# Intérprete: Ejemplos

- Usualmente visto en lenguajes de programación
- Fácil de usar, flexible y portable
  - Ej: *py* lo usan en todos lados
- Intérpretes de *py/js*
- Bash



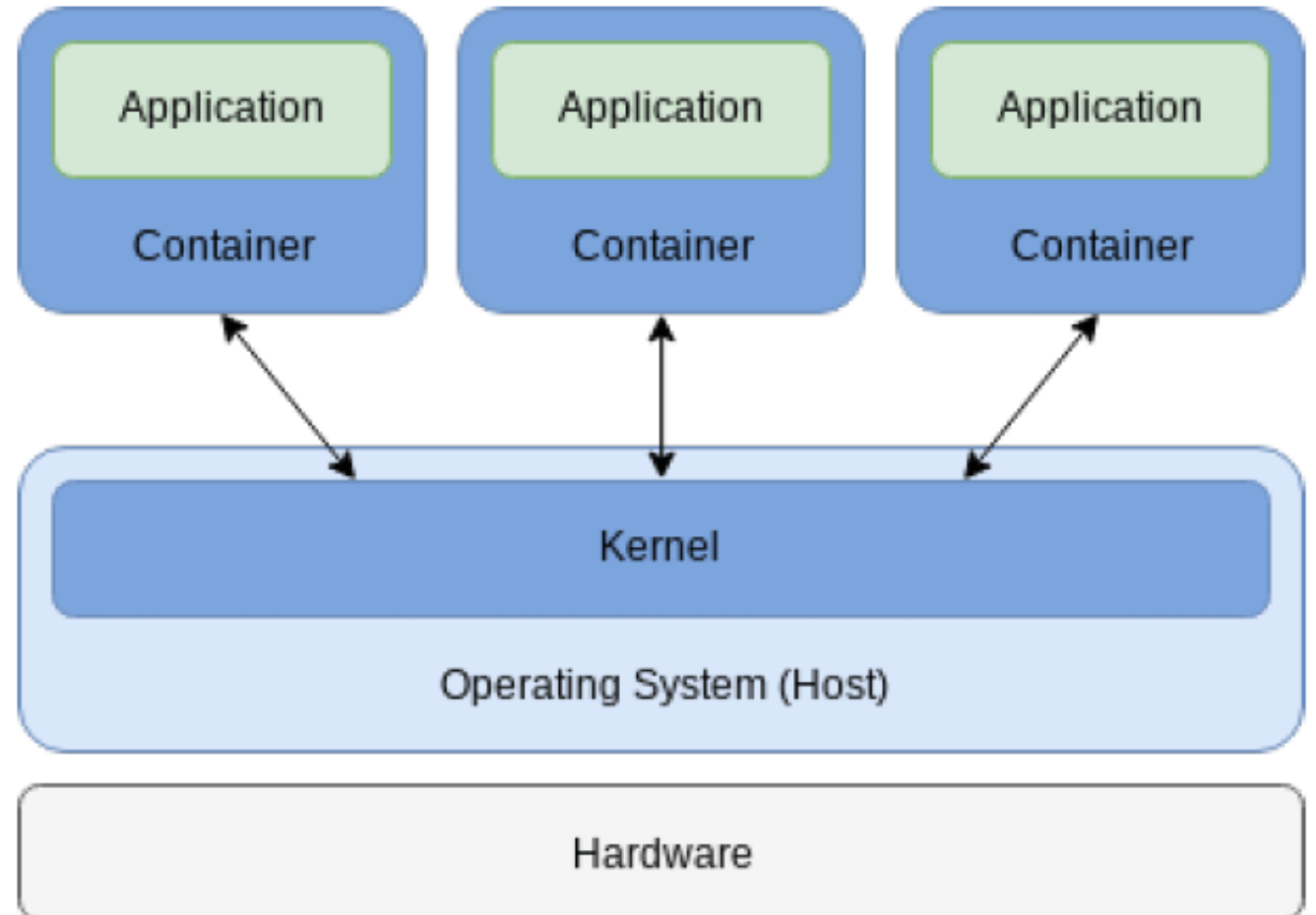
# Intérprete: Ejemplos

- Usualmente visto en lenguajes de programación
- Fácil de usar, flexible y portable
  - Ej: *py* lo usan en todos lados
- Intérpretes de *py/js*
- Bash



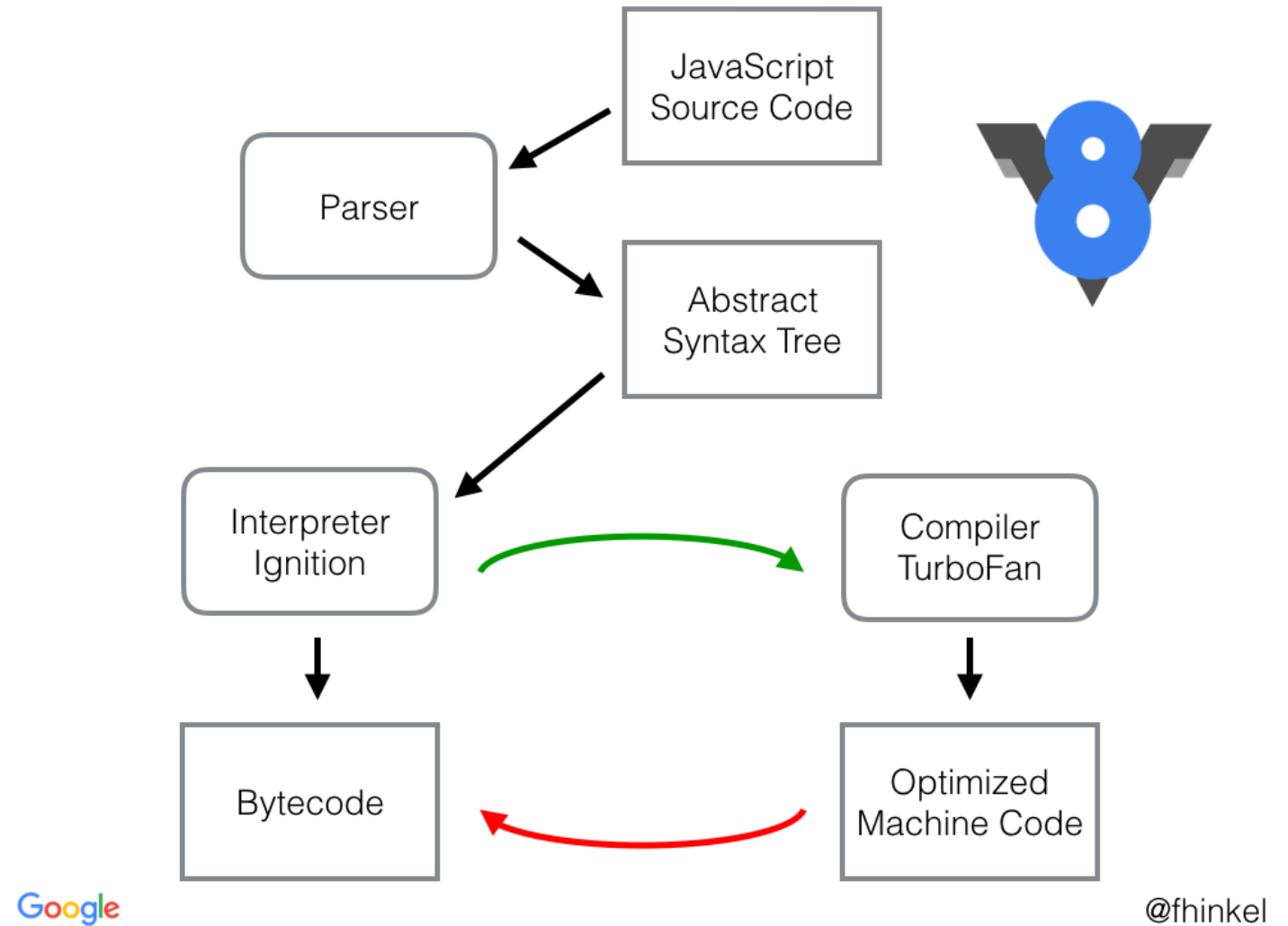
# Intérprete: Código móvil

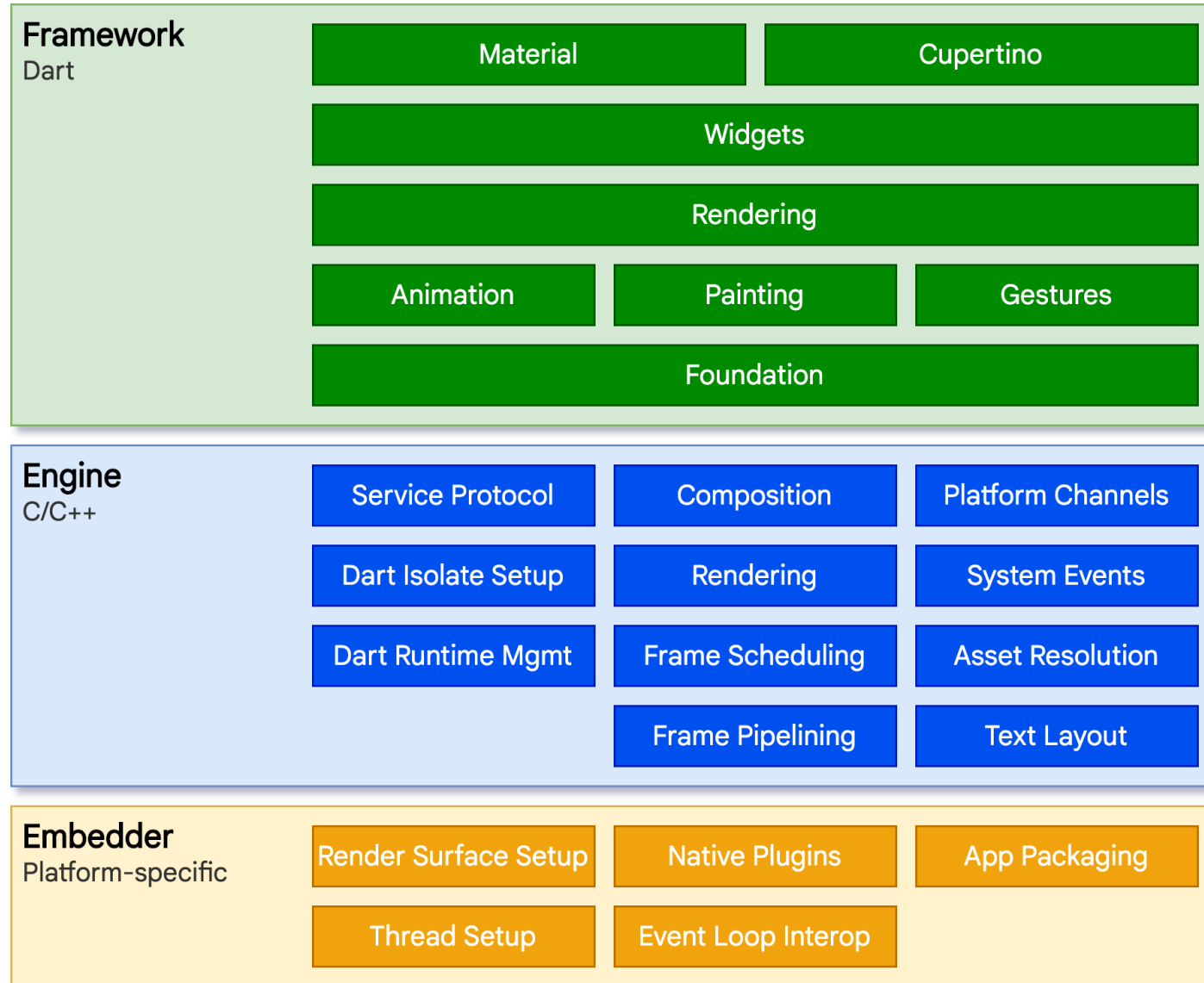
- Un elemento de datos se transforma dinámicamente en un componente de procesamiento de datos
- Software que puede ejecutarse en múltiples nodos sin instalar o compilar
- Transmitido por red
- Lenguajes que usan: JS, Java, Dart

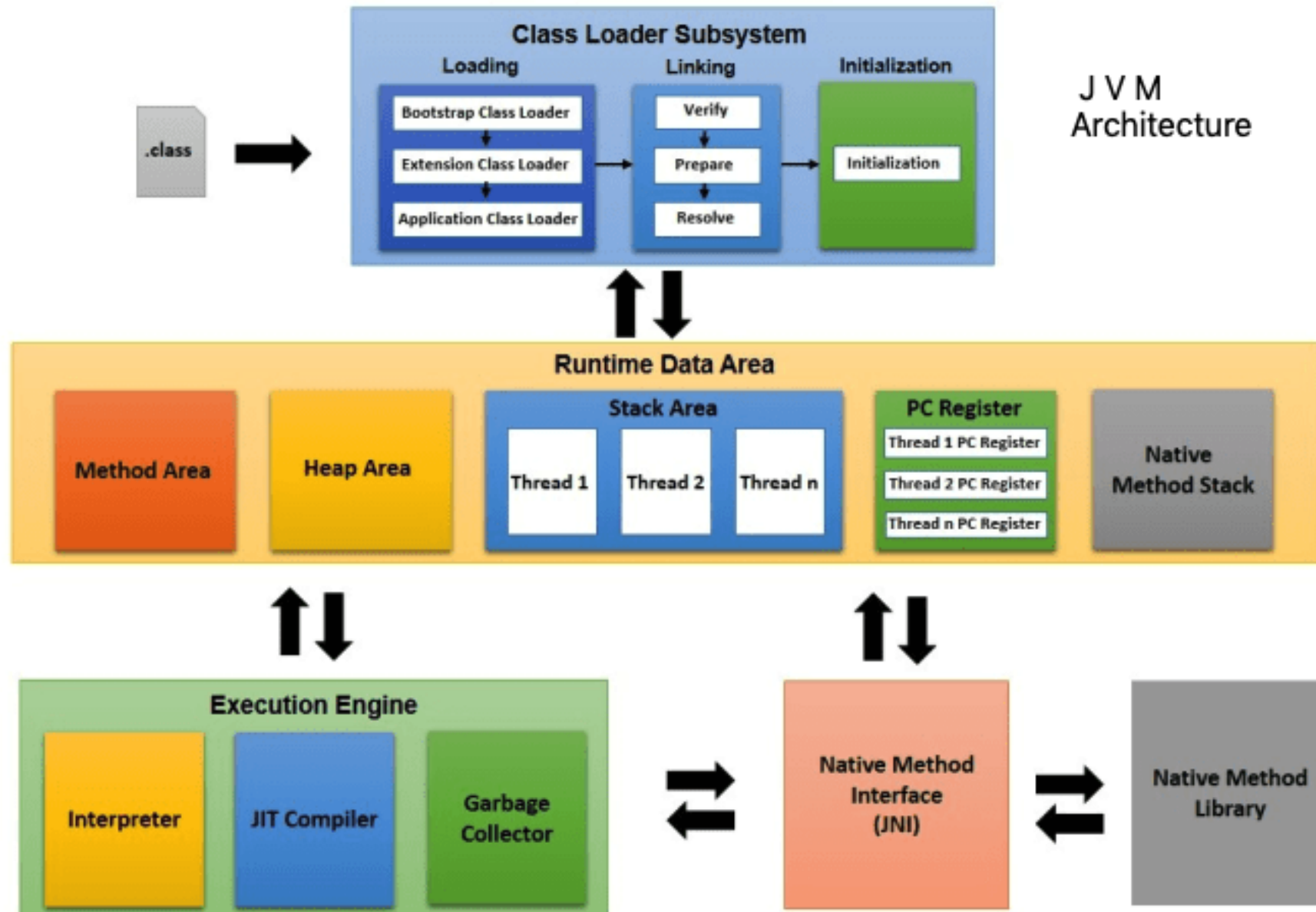


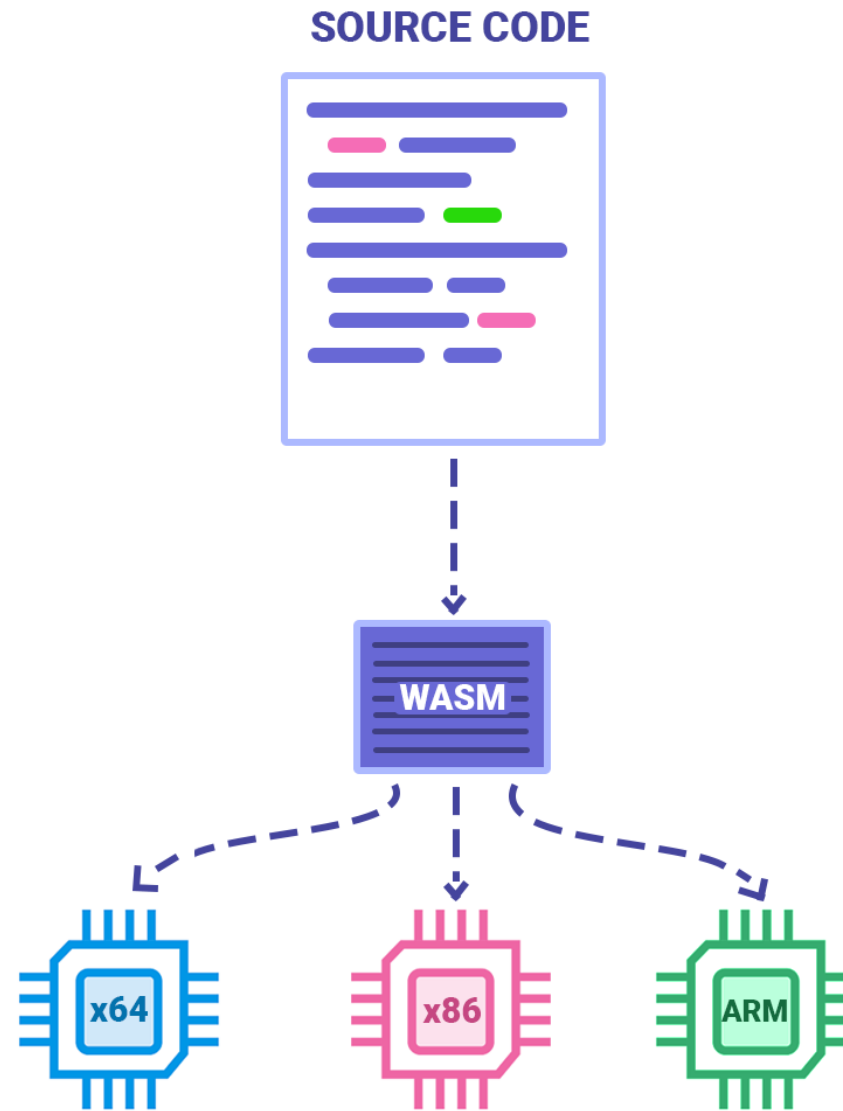
# Código móvil

- Datos: Código como datos (bundles, archivos)
- Topología: Vía red
- Variantes
  - Código bajo demanda
  - Evaluación remota
  - Agentes móviles (ej: agentes de monitoreo)
- *Containers* cuando se despliegan a varios tipos de sistemas





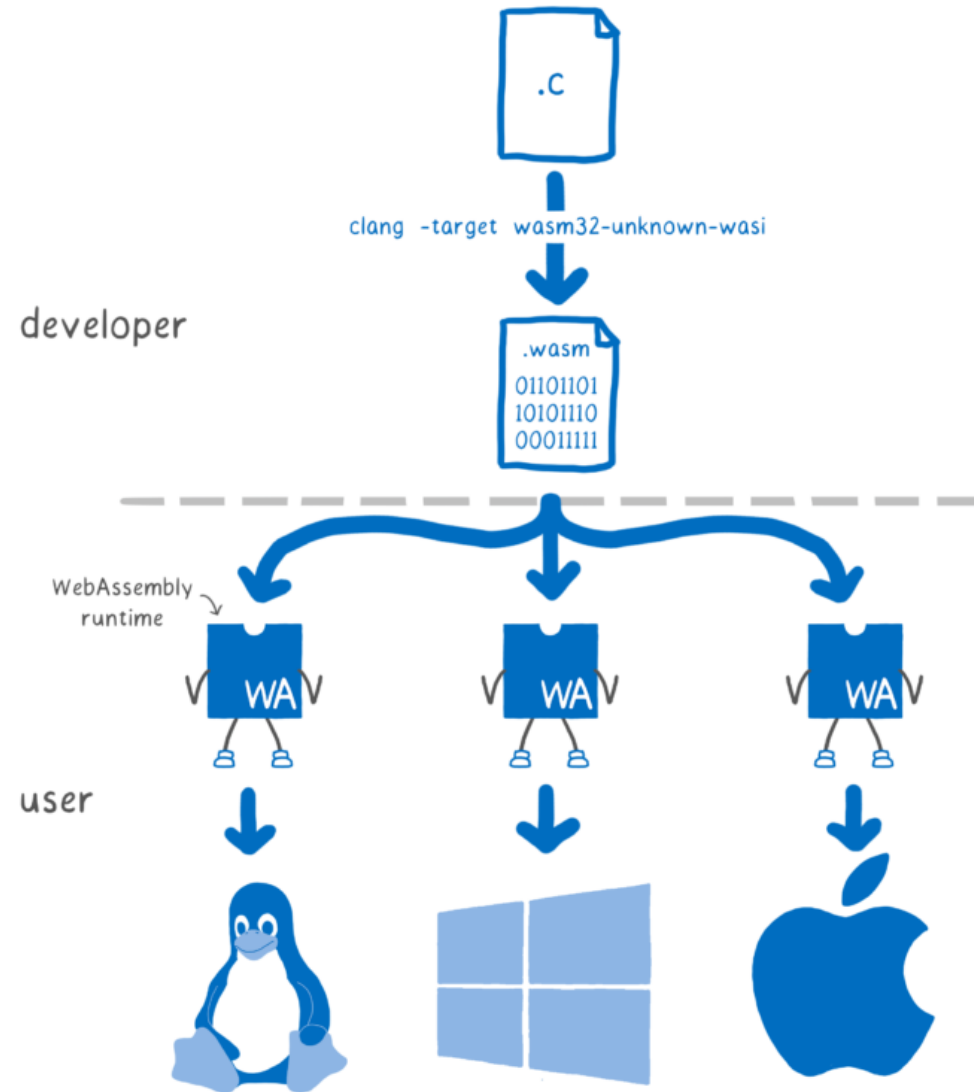




WASM

# Código móvil: Pros

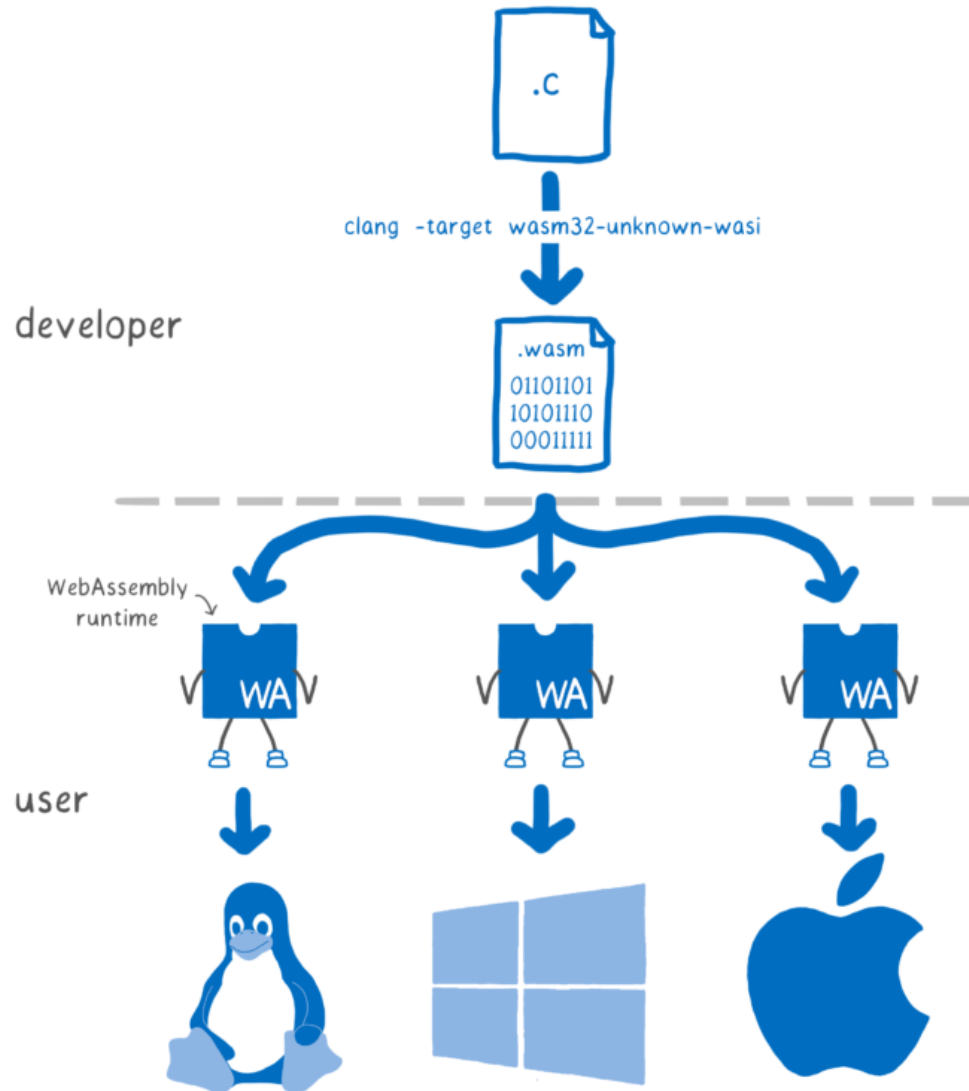
- Portabilidad suprema
- Contenido mejorado
  - Dinámico
  - Interactivo
- Reduce carga de transmisión en ciertos escenarios





# Código móvil: Contras

- Riesgo de seguridad: ¿De dónde viene ese código?
  - [https://owasp.org/www-community/vulnerabilities/Unsafe\\_Mobile\\_Code](https://owasp.org/www-community/vulnerabilities/Unsafe_Mobile_Code)
- Privacidad
- Trabajo en compatibilidades
  - 1 base en cada plataforma
- Performance: Más intermediarios

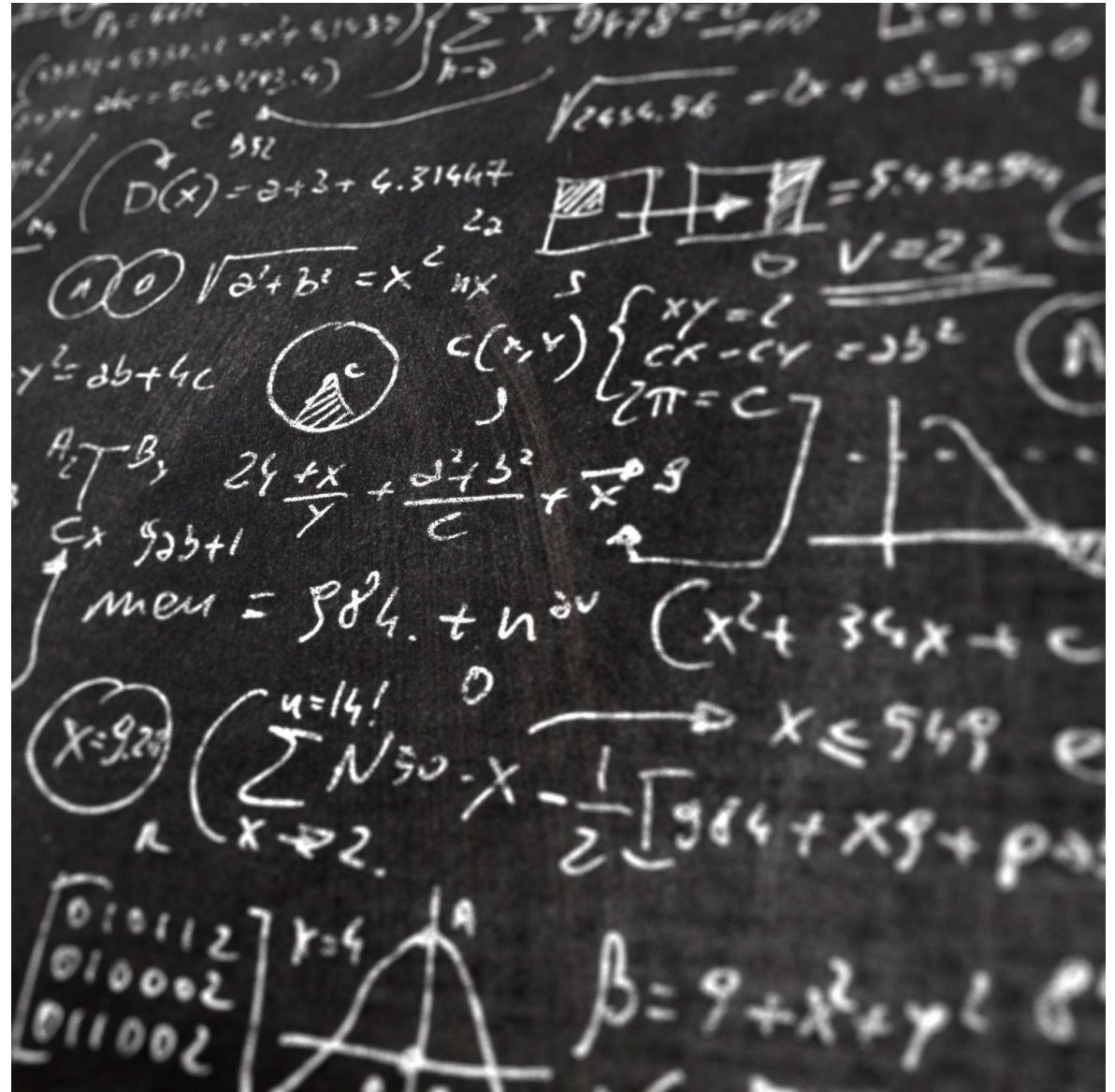


# Máquinas virtuales y Código móvil

- Puede pasar que el código móvil corre sobre alguna máquina virtual
- Puede ayudar en **seguridad**
- Ayuda aún más a **portabilidad**

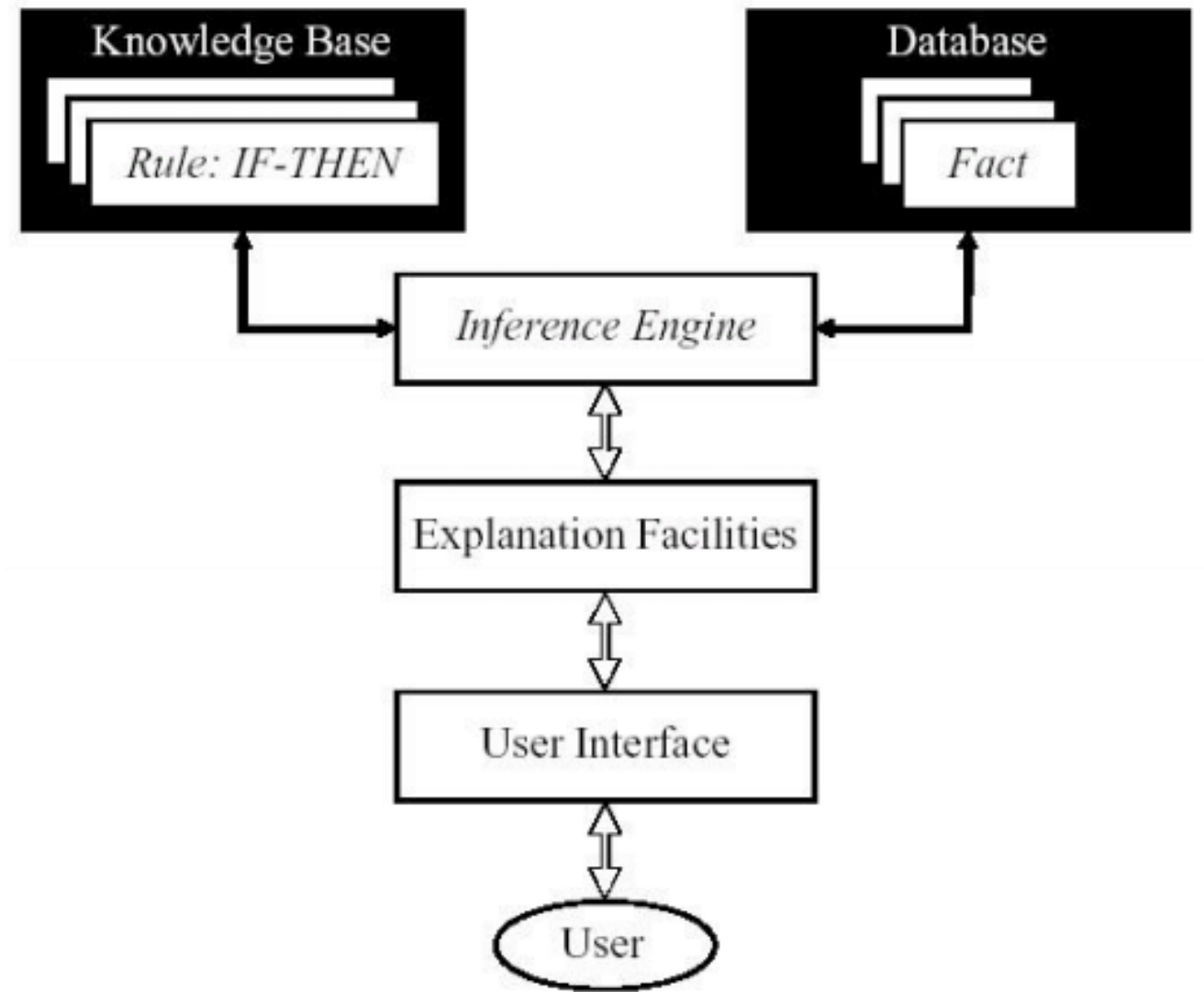
# Memoria compartida

- Un **motor de inferencia** parsea las entradas de los usuarios y determina si es un "hecho", una "regla" (y los añade a la base de conocimiento) o una "consulta" (y la resuelve)
- Los hechos y reglas se añaden a la base de conocimiento
- Las **consultas se ejecutan sobre la base de conocimiento** para evaluar las reglas que se puedan aplicar y resolver la consulta



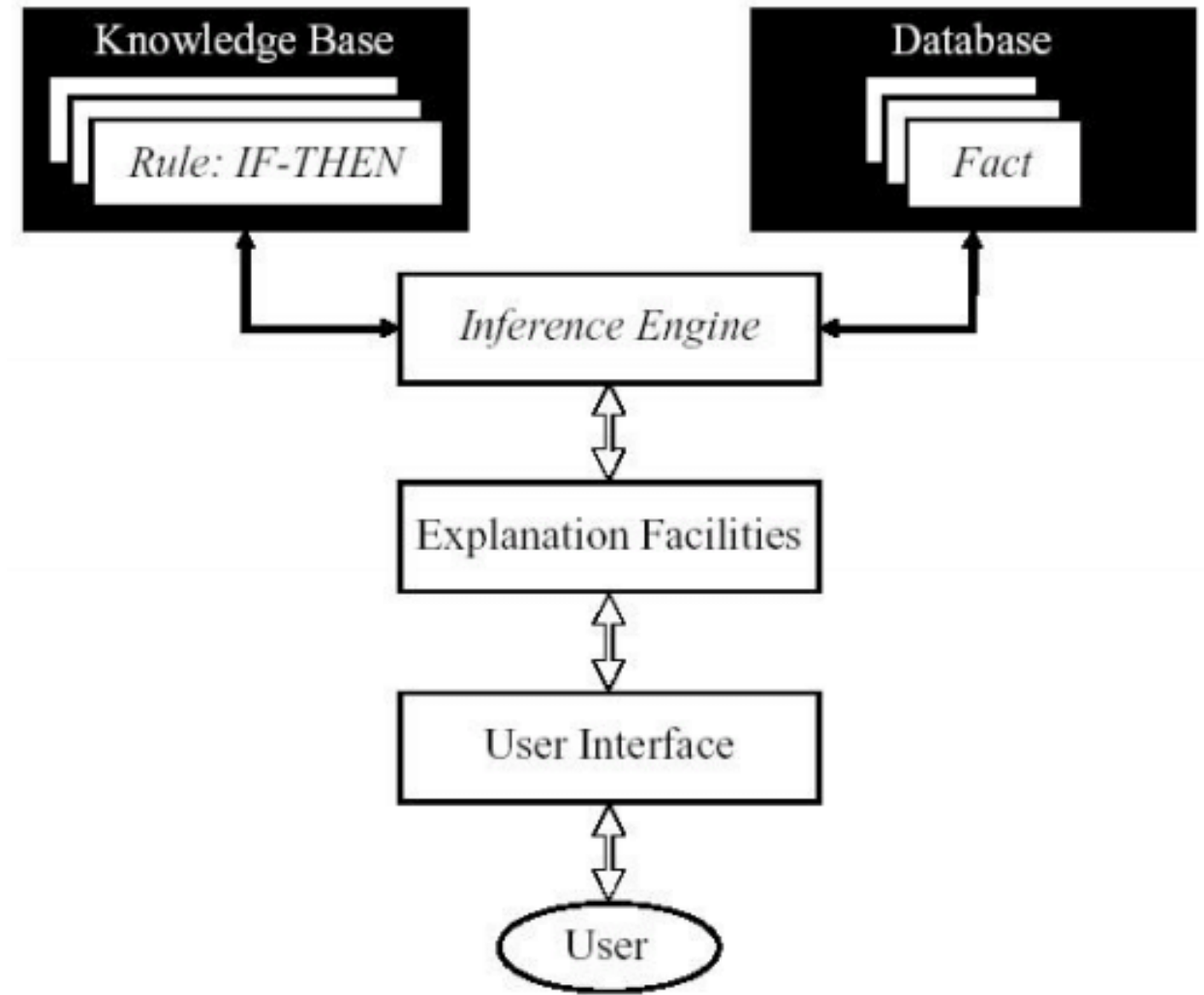
# Memoria compartida: Basado en reglas

- La “conducta” de la aplicación se extiende **mediante reglas**
  - If/else, arboles de decisión, otros...
- IPC mediante almacenes de datos o buffers
- Riesgo: Al aumentar el número de reglas es muy difícil entender las interacciones entre las reglas



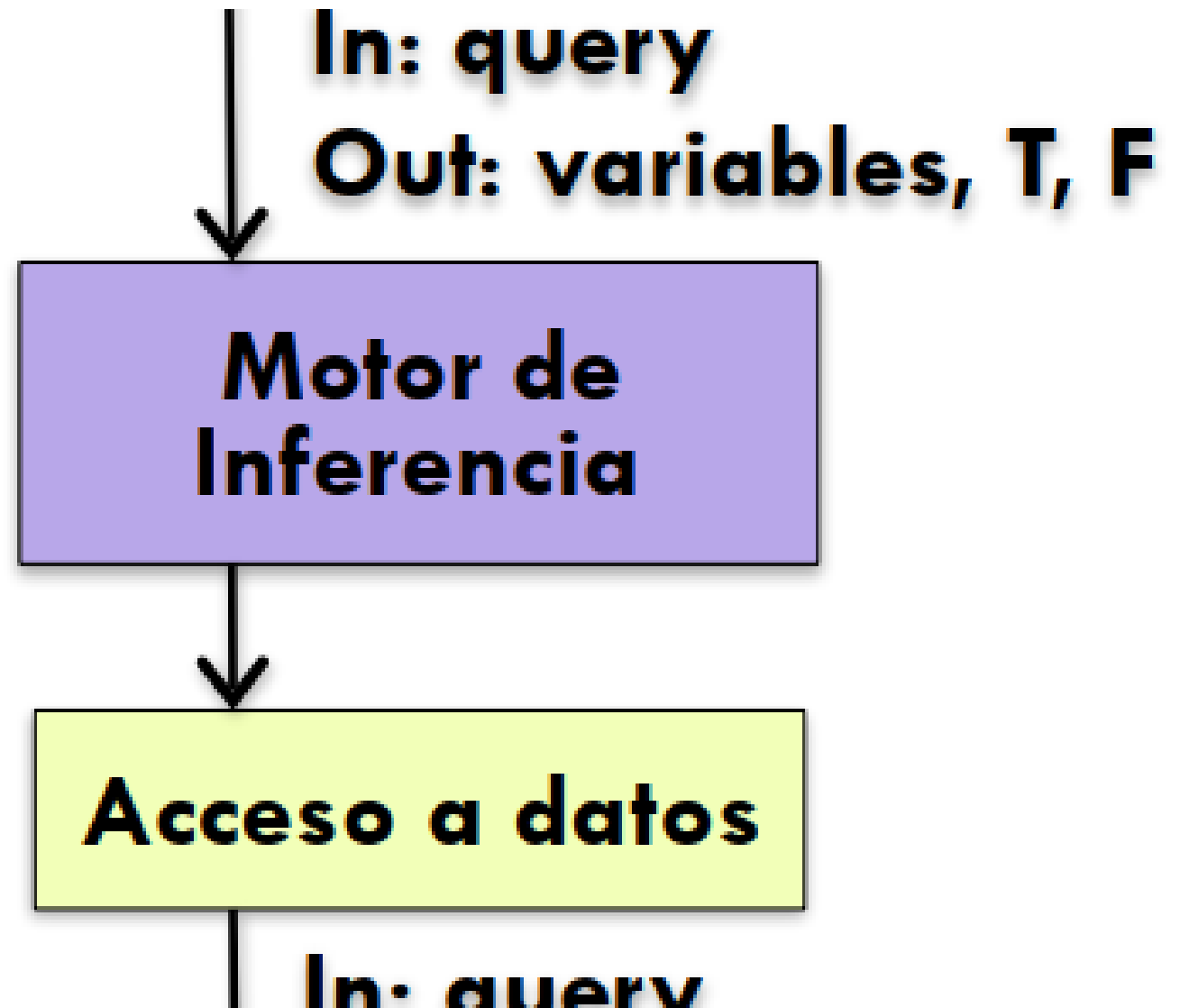
## Basado en reglas: Características

- Componentes
  - Interfaz de usuario
  - Motor de inferencia
  - Base de conocimiento
- Conectores
  - Componentes fuertemente interconectados vía llamadas a procedimientos y/o llamadas a memoria compartida
- Elementos de datos
  - Hechos y consultas



## Basado en reglas: Ejemplos

- Sistemas de tráfico
  - Memoria compartida
  - Reglas de tránsito
  - Manejo de concurrencia
- Sistemas de seguridad
- Sistemas expertos



# Basado en reglas: Ejemplo

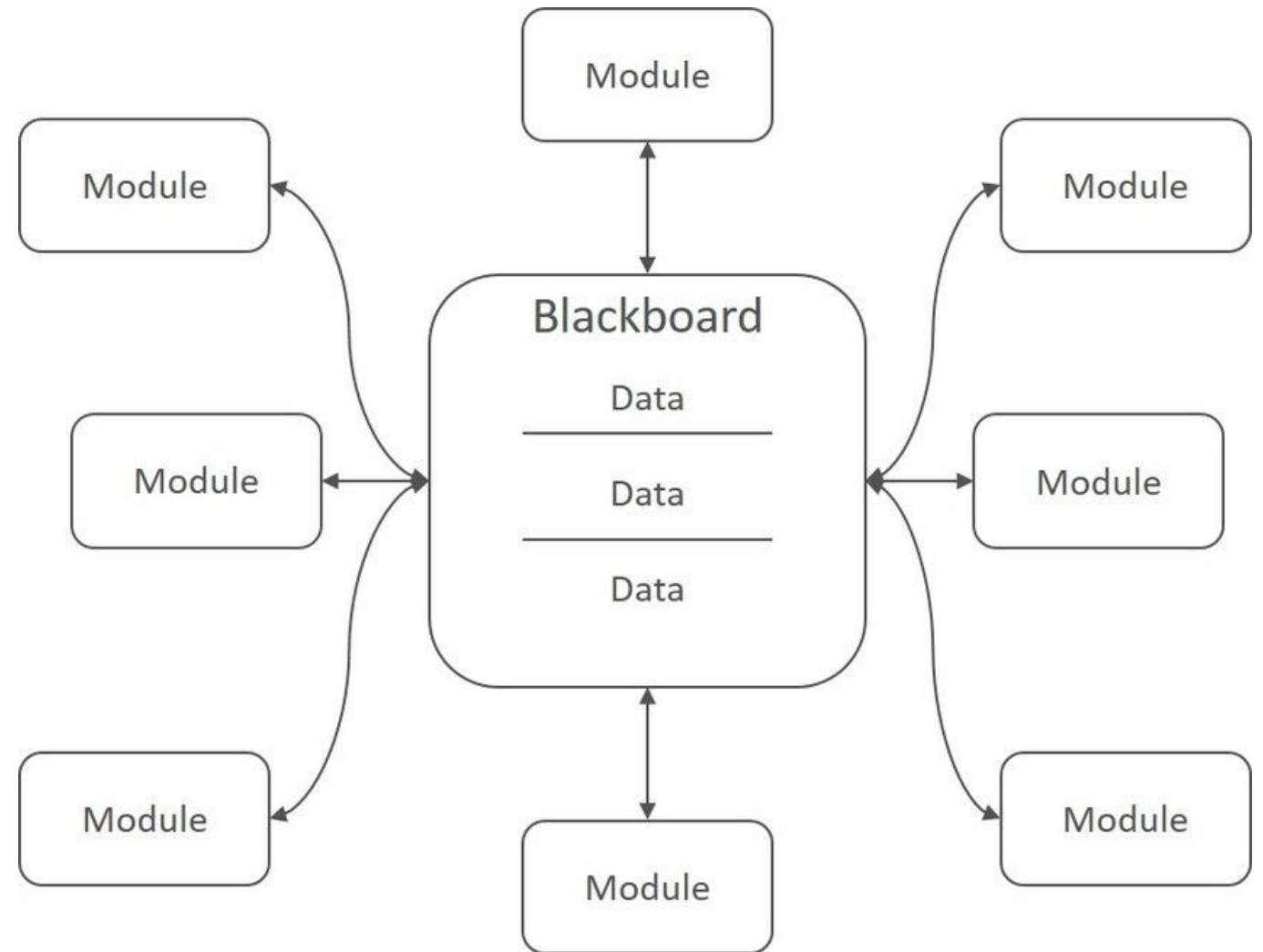
```
(defrule welcome-toddlers "Saludo a niños!"  
  (person {age < 3})  
  =>  
  ((System.out) println "Hola, pequeño!"))
```

```
(person (age ?a) (firstName ?f) (lastName ?l))
```

```
(assert (person(age 12)(firstName maria)(lastName lopez)))  
(assert (person(age 2)(firstName lucas)(lastName martin)))  
(assert (person(age 5)(firstName pedro)(lastName perez)))
```

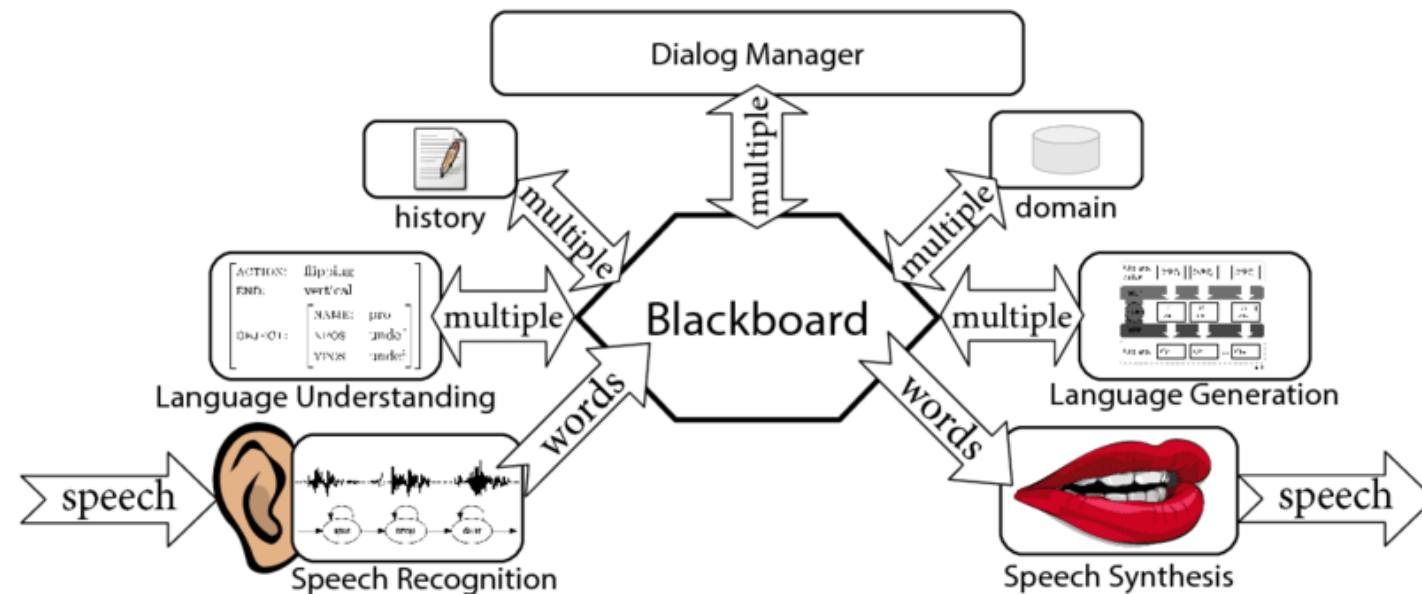
# Memoria compartida: Pizarra

- Un **espacio de datos central consultado y modificado por fuentes de conocimiento**
  - Fuentes de conocimiento aportan solución a un problema tomando partes de este
  - **Blackboard**: se organiza la data para ser accesible
  - Control: supervisa el resultado
- Análogo: Grupo de científicos modificando una pizarra
- Riesgo: Al aumentar el número de reglas es muy difícil entender las interacciones entre estas



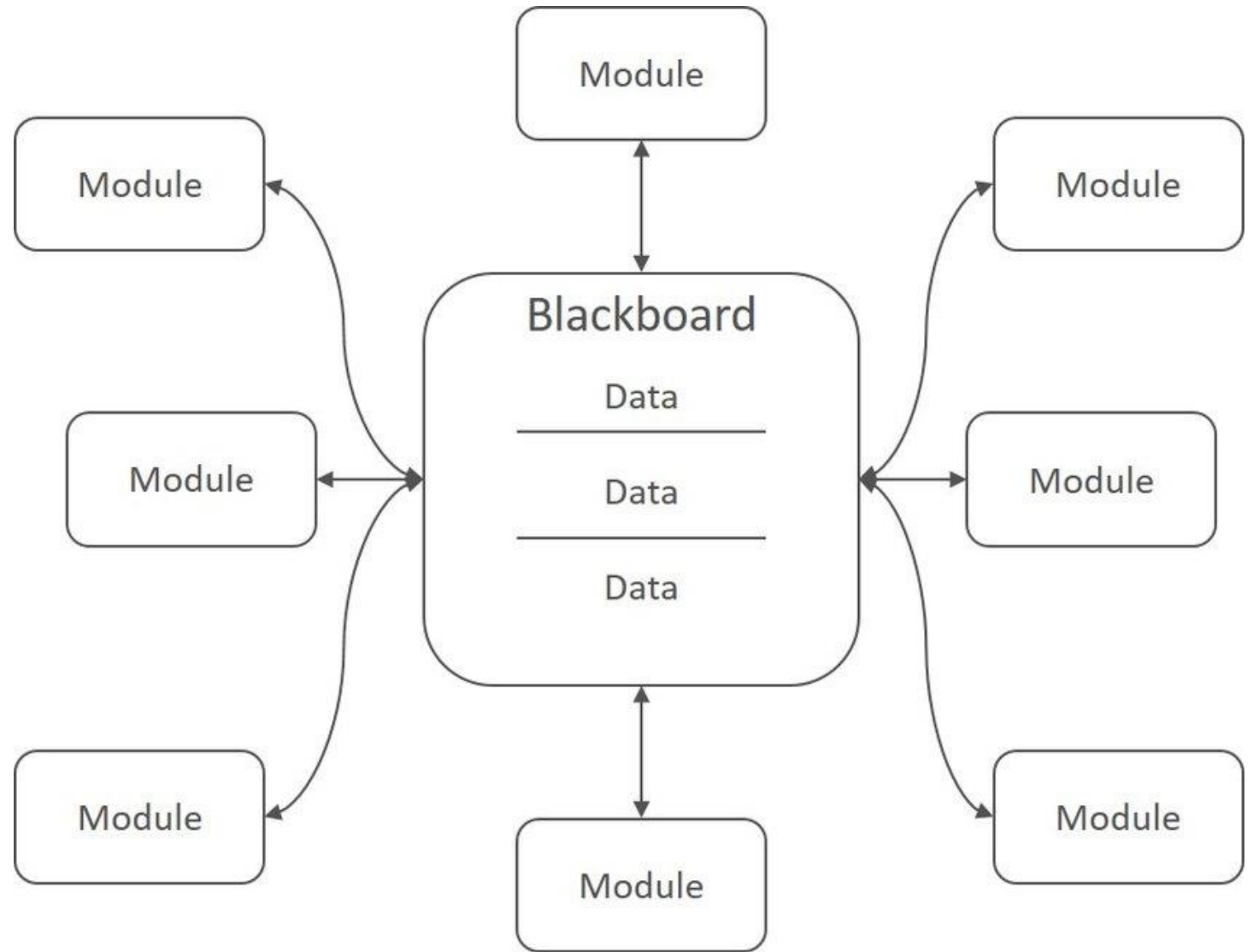


## Pizarra: Ejemplos



# Pizarra: Balance

- Pros
  - Posibilidad de problemas complejos
  - Arquitectura colaborativa
- Cons
  - Escaso uso
  - Complejo de implementar correctamente
  - Puede complejizar búsquedas si no se controla



# Estrategias para varios patrones

Bass, Kazman

| Pattern                          | Modifiability               |                          |                 |               |                      |                     |                             |                          |                          |                     |
|----------------------------------|-----------------------------|--------------------------|-----------------|---------------|----------------------|---------------------|-----------------------------|--------------------------|--------------------------|---------------------|
|                                  | Increase Cohesion           |                          | Reduce Coupling |               |                      |                     | Defer Binding Time          |                          |                          |                     |
|                                  | Increase Semantic Coherence | Abstract Common Services | Encapsulate     | Use a Wrapper | Restrict Comm. Paths | Use an Intermediary | Raise the Abstraction Level | Use Runtime Registration | Use Startup-Time Binding | Use Runtime Binding |
| Layered                          | X                           | X                        | X               |               | X                    | X                   | X                           |                          |                          |                     |
| Pipes and Filters                | X                           |                          | X               |               | X                    | X                   |                             |                          | X                        |                     |
| Blackboard                       | X                           | X                        |                 |               | X                    | X                   | X                           | X                        |                          | X                   |
| Broker                           | X                           | X                        | X               |               | X                    | X                   | X                           | X                        |                          |                     |
| Model View Controller            | X                           |                          | X               |               |                      | X                   |                             |                          |                          | X                   |
| Presentation Abstraction Control | X                           |                          | X               |               |                      | X                   | X                           |                          |                          |                     |
| Microkernel                      | X                           | X                        | X               |               | X                    | X                   |                             |                          |                          |                     |
| Reflection                       | X                           |                          | X               |               |                      |                     |                             |                          |                          |                     |



# Estilos y Patrones IV

---

IIC2173 - Arquitectura de sistemas de software

Departamento de Ciencia de la Computación

Escuela de Ingeniería – PUC

Nicolás Acosta – Gerardo Crot

| Estilo                                   | Procesamiento                                                                                                                                                                                 | Cuándo si?                                                                                                     | Cuándo no?                                                                                                                | Impacto en Calidad                  |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| <b>Influencia-<br/>do x<br/>lenguaje</b> | <ul style="list-style-type: none"> <li>- El orden del procesamiento <b>es conocido</b></li> <li>- Los componentes <b>esperan la respuesta</b> de otros componentes para continuar.</li> </ul> |                                                                                                                |                                                                                                                           |                                     |
| <b>Main y subrutinas</b>                 | - El problema puede des-componerse en problemas pequeños                                                                                                                                      | Aplicación es pequeña y simple                                                                                 | Se requieren estructuras de datos complejas.<br>Se modificará                                                             | Performance (+)<br>Reusabilidad (+) |
| <b>Orientado a objetos</b>               | - El problema puede mode-larse via entidades que interactúan.                                                                                                                                 | Hay un mapeo claro entre entidades del pro-blema y objetos. Estructuras de datos complejas e interrelacionadas | Aplicaciones distribui-das en redes hetero-géneas. Componentes fuertemente indepen-dientes. Se requiere alto performance. | Flexibilidad (+)<br>Integridad (+)  |



| Estilo             | Procesamiento                                                                                                                                                                                           | Cuándo si?                                                                                           | Cuándo no?                                                                                                                          | Impacto en Calidad                    |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| Capas              | <ul style="list-style-type: none"> <li>- Las tareas se pueden dividir en capas de abstracción.</li> <li>- El orden del procesamiento <b>es conocido</b></li> </ul>                                      |                                                                                                      |                                                                                                                                     |                                       |
| Máquinas virtuales | - La data tiene 2 partes: Información y control que indica cómo procesar la información.                                                                                                                | Una única capa sirve de servicio para otras aplicaciones.<br>La interfaz de la capa base es estable. | Se requieren varias capas.<br>Los datos se acceden por varias capas.                                                                | Portabilidad (+)                      |
| Cliente Servidor   | <ul style="list-style-type: none"> <li>- El servidor centraliza y ejecuta un proceso para varios clientes.</li> <li>- El cliente hace pedidos al servidor y espera la respuesta para seguir.</li> </ul> | Puede centralizarse algunas tareas.<br>El procesamiento del cliente es limitado.                     | La centralización es punto único de falla. Ancho de banda limitado.<br>La capacidad del cliente compite o excede a la del servidor. | Escalabilidad (+)<br>Flexibilidad (+) |

| Estilo                      | Procesamiento                                                                                                                                                                                                                                                                                       | Cuándo si?                                                        | Cuándo no?                                                        | Impacto en Calidad                                                           |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------|
| Flujo de datos              | <ul style="list-style-type: none"> <li>- Datos de entrada/salida bien definidos</li> <li>- Salida producida al procesar en forma secuencial el input, <b>independiente del tiempo</b> (no hay espera, <b>send-and-forget</b>)</li> </ul>                                                            |                                                                   |                                                                   |                                                                              |
| En lotes, <u>secuencial</u> | <ul style="list-style-type: none"> <li>- La lectura y procesamiento de un conjunto de datos (lote) genera una sola salida.</li> <li>- La entrada se procesa por una serie de transformaciones conectadas secuencialmente.</li> </ul>                                                                | Problema dividible en una secuencia de pasos                      | Se requiere: interactividad, concurrencia o acceso a datos random | Reusabilidad (+)<br>Modificabilidad (+)<br><b>Performance no importa (.)</b> |
| Pipe and filter             | <ul style="list-style-type: none"> <li>- Transformaciones sobre un <b>stream continuo</b> de datos</li> <li>- Transformaciones incrementales</li> <li>- <b>Una transformación sólo puede empezar cuando la anterior terminó</b></li> <li>- Las transformaciones pueden ser concurrentes.</li> </ul> | Idem.<br>Filtros reusables<br>Estructuras de datos serializa-bles | Se requiere interacción entre componentes                         | Escalabilidad (+)<br>Performance (+)                                         |

| Estilo                    | Procesamiento                                                                                                                                                                    | Cuándo si?                                                                                                    | Cuándo no?                                                                                                    | Impacto en Calidad                                       |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <b>Memoria compartida</b> | - El elemento central es el <b>almacenamiento, representación, gestión y recuperación de grandes cantidades de datos interrelacionados que deben graduarse por largo tiempo.</b> |                                                                                                               |                                                                                                               |                                                          |
| <b>Pizarra</b>            | - Programas independientes se comunican únicamente a través de un repositorio global.                                                                                            | Los cálculos usan/ cambian datos compartidos.<br>El orden de procesamiento se define on runtime, según datos. | Programa accede a partes independientes de datos.<br>La interfaz de datos puede cambiar.<br>Interacción entre | Escalabilidad(+)<br>Modificabilidad(+)                   |
| <b>Basado en reglas</b>   | - Usa hechos/reglas de una KB para responder una consulta.                                                                                                                       | Los datos y preguntas del problema se pueden modelar como reglas de inferencia simples.                       | El número de reglas es grande.<br>Hay interacción entre reglas.<br>Requiere gran performance.                 | Escalabilidad(+)<br>Modificabilidad(+)<br>Performance(-) |



| Estilo              | Procesamiento                                                                                          | Cuándo si?                                                                                                                               | Cuándo no?                                                                                                                        | Impacto en Calidad                                                   |
|---------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| <b>Intérprete</b>   | <b>-Se enfoca en la interpretación <i>dinámica</i> (en tiempo de ejecución) de comandos explícitos</b> |                                                                                                                                          |                                                                                                                                   |                                                                      |
| <b>Intérprete</b>   | <b>-El intérprete parsea y ejecuta un stream de entrada y actualiza su estado.</b>                     | <b>Se requiere conducta dinámica y mucha personalización de usuario.</b>                                                                 | <b>Se requiere alto performance</b>                                                                                               | <b>Portabilidad (+)<br/>Modificabilidad (+)<br/>Performance (-)</b>  |
| <b>Código móvil</b> | <b>-Se mueve el código a un host que lo ejecuta.</b>                                                   | <b>Es más eficiente mover el “proceso” cerca de los datos que al revés. Se customiza código local con código externo, dinámicamente.</b> | <b>No se puede garantizar la seguridad del código móvil, o tiene acceso restringido. Se requiere fuerte control de versiones.</b> | <b>Modificabilidad(+)<br/>Extensibilidad (+)<br/>Performance (+)</b> |

| <b>Estilo</b>               | <b>Procesamiento</b>                                                                                             | <b>Cuándo si?</b>                                                                                        | <b>Cuándo no?</b>                                                         | <b>Impacto en Calidad</b>                                         |
|-----------------------------|------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Invocación implícita</b> | <b>- Se caracteriza por llamadas indirectas e implícitas, tales como, respuestas a notificaciones o eventos.</b> |                                                                                                          |                                                                           |                                                                   |
| <b>Publisher-Subscribe</b>  | <b>Un publicador difunde mensajes a subscriptores.</b>                                                           | <b>Componentes débilmente acoplados. Los datos de suscripción son pequeños y fáciles de transportar.</b> | <b>No hay middleware para soportar muchos datos.</b>                      | <b>Escalabilidad (+)<br/>Flexibilidad (+)<br/>Performance (+)</b> |
| <b>Basada en eventos</b>    | <b>Componentes independientes emiten y reciben eventos asíncronamente.</b>                                       | <b>Componentes concurrentes e independientes. Componentes heterogéneos, distribuidos y en red.</b>       | <b>Se requieren garantías de procesamiento de eventos en tiempo real.</b> | <b>Escalabilidad (+)<br/>Flexibilidad (+)<br/>Performance (+)</b> |

| <b>Estilo</b>       | <b>Procesamiento</b>                                                              | <b>Cuándo si?</b>                                                                                                                                                               | <b>Cuándo no?</b>                                                                                                                                        | <b>Impacto en Calidad</b>                                         |
|---------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Peer-To-Peer</b> | <b>Los peers guardan estado (datos) y actúan como clientes y como servidores.</b> | <b>Los peers se distribuyen en redes que pueden ser heterogéneas e independientes.<br/>Se requiere robustez ante fallas independientes.<br/>Se requiere alta escalabilidad.</b> | <b>No se puede garantizar la confiabilidad de peers independientes.<br/>Hay nodos designados para descubrir recursos que pueden no estar disponibles</b> | <b>Escalabilidad (+)<br/>Flexibilidad (+)<br/>Performance (+)</b> |